Fluenzic English Enhancement App

Mini project report submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering

Submitted by

Eveline Jeson (FIT22CS072)
Fahmitha P A (FIT22CS075)
Irine Theresa Antony (FIT22CS097)
Merin Joe (FIT22CS124)

Federal Institute of Science And Technology (FISAT)®
Angamaly, Ernakulam

Affiliated to

APJ Abdul Kalam Technological University
CET Campus, Thiruvananthapuram

April 2025

FEDERAL INSTITUTE OF SCIENCE AND TECHNOLOGY (FISAT)
Mookkannoor(P.O), Angamaly-683577

CERTIFICATE

This is to certify that the Mini project report for the project entitled "**Fluenzic English Enhancement App**" is a bonafide report of the project presented during VIth semester (CSD334 - Mini Project) by **Merin Joe(FIT22CS124), Eveline Jeson(FIT22CS072), Fahmitha P A(FIT22CS075) and Irine Theresa Antony(FIT22CS097)**, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology (B.Tech) in Computer Science & Engineering during the academic year 2024-25.

Dr.Prasad J C	**Dr.Hema Krishnan**	**Dr.Paul P Mathai**
Project Coordinator	Project Guide	Head of the Department

ABSTRACT

This project addresses the challenge of achieving English fluency through *Fluenzic*, an AI-driven platform integrating real-time conversational practice, grammar correction, and contextual vocabulary enhancement. Traditional tools focus on isolated skills, leaving a gap in adaptive, real-world communication training. *Fluenzic* bridges this gap using **Microsoft Azure Speech-to-Text** for voice transcription, **OpenAI GPT-4** for context-aware feedback, and a **Word2Vec model** to power interactive vocabulary games.

The system offers AI-powered voice calls simulating natural conversations, writing tools with grammar recommendations, and gamified quizzes where users identify target words from AI-generated synonyms. For instance, the Word2Vec model generates synonyms (e.g., "happy" for "joyful"), and users guess the correct word, reinforcing retention through contextual learning. The platform's novelty lies in its unified approach to speaking, writing, and vocabulary development, dynamically adapting to individual proficiency levels. By prioritizing accessibility and GDPR-compliant security, *Fluenzic* provides an affordable solution for non-native speakers to gain confidence in professional and social communication. This project demonstrates how modern language processing can make language learning accessible to everyone. .

Contribution by Author

My primary focus was on designing and implementing the Quiz Module and User Progress Dashboard to create a seamless learning experience. I started by mapping out the user journey, ensuring the interface was intuitive and accessible across devices. For the Quiz Module, I prioritized clean layouts and interactive elements to keep users engaged during assessments. A key challenge was integrating real-time data tracking, which I addressed by connecting the frontend to MongoDB. This allowed us to store quiz scores, track performance trends, and generate progress reports. To make the data actionable, I built dynamic visualizations—like progress charts and streak counters—using MongoDB aggregation pipelines. These tools update in real time, giving users immediate feedback on their learning journey. My goal was to transform raw data into meaningful insights, helping learners stay motivated by visualizing their growth.

Eveline Jeson

Contribution by Author

I led the development of the Call Module and the core Authentication System, as well as the Admin Dashboard. For the voice call feature, I designed a simple yet functional interface that supports real-time communication. Security was a top priority, so I implemented JWT-based authentication and integrated Firebase to enable Google and email logins. The most complex task was building the WebSocket pipeline for voice processing: I configured Azure Speech-to-Text to transcribe user input, then routed the text to GPT-4 for grammar corrections before sending feedback via RTClient. This ensured low-latency interactions, mimicking natural conversation. On the admin side, I created role-based access controls and tools for managing users and content. By optimizing MongoDB queries, I enabled admins to filter usage analytics, delete inactive accounts, and monitor system performance efficiently. My work aimed to balance user convenience with robust backend security.

Fahmitha P A

Contribution by Author

I designed the Interactive Word Game and Vocabulary Builder to help users expand their English vocabulary in a fun, engaging way. The word game focuses on contextual learning—for example, players match words like "proactive" with their synonyms, such as "initiative," based on sentences pulled from real-world scenarios. To make the game adaptive, I trained a custom Word2Vec model using relevant datasets. This allowed the system to generate synonyms that aren't just dictionary-perfect but also contextually relevant. I structured MongoDB collections to track game scores. The minimalistic UI was intentional: I wanted users to focus on the words and their meanings without distractions. My goal was to make learning feel less like a chore and more like a puzzle that rewards curiosity.

Irine Theresa Antony

Contribution by Author

My primary contribution to this project focused on enhancing users' writing skills through AI-driven analysis and feedback. I developed the Writing Enhancement Module, where I integrated Azure OpenAI's API to provide real-time spelling, grammar and style corrections. Users write in a rich-text editor, and the system instantly detects errors, displays mistakes, and offers contextual explanations alongside corrections.

Additionally, I contributed to the Daily Vocabulary Module, where user-generated writing is analyzed for vocabulary usage. This system evaluates whether the words used are contextually appropriate and suggests better alternatives, aiding in vocabulary building. For instance, if a user misuses a word or uses basic vocabulary, the system suggests more precise or advanced alternatives while providing explanations.

To support these features, I designed MongoDB schemas to efficiently store and track user progress, writing drafts, detected errors, and vocabulary usage trends. This data-driven approach allows the system to adapt to each user's learning curve, reinforcing long-term improvement.

My goal was to create an interactive and intelligent writing assistant that not only corrects mistakes but also fosters deeper learning and skill enhancement.

Merin Joe

ACKNOWLEDGMENT

We express our sincere gratitude to all who supported the development of this project. We express out sincere thanks to our guide Dr. Hema Krishnan, whose mentorship guided us through technical challenges and encouraged us to refine our approach iteratively. Her availability for detailed discussions and constructive feedback during critical stages was instrumental to our progress. We acknowledge Mr Jacob Thomas V, Principal, Federal Institute of Science And Technology,Angamaly for providing access to institutional resources and laboratory facilities. Special thanks to the CSE department faculty members, particularly Dr Paul P Mathai,HOD CSE department, for their academic guidance and flexibility during the mini-project period. The Microsoft Founders Hub program deserves recognition for granting Azure Cloud credits that made our AI/ML implementations financially feasible. This support allowed us to integrate advanced cognitive services that would otherwise have been beyond our student budget. To our dedicated team members - your persistence in debugging complex integrations and willingness to cross-learn unfamiliar technologies kept the project moving forward. This collaborative spirit transformed individual contributions into a cohesive solution.

Contents

List of Figures

List of Tables

1 Introduction **1**

2 Literature Review **4**
2.1 Related Work 4
2.2 Comparison of Related Works 4
2.3 Proposed System 5

3 Design Methodologies **7**
3.1 Software Requirement Specifications 7
3.1.1 Project Purpose 7
3.1.2 User Requirements 7
3.1.3 System Requirements 7
3.1.4 Functional Requirements 8
3.1.5 Non-Functional Requirements 8
3.2 Software Design Document 9
3.2.1 System Features 9
3.2.2 System Architecture Design 9
3.2.3 Constraints 10
3.2.4 Application Architecture Design 10
3.2.5 API Design 12
3.2.6 Database Design 17
3.3 Use Case Diagrams/Data Flow Diagrams 20
3.3.1 Data Flow Diagram 20
3.4 Algorithms 22
3.4.1 Login and Signup Algorithm 22
3.4.2 Voice Call Algorithm 22
3.4.3 Audio Handling and Response Processing Algorithm 23
3.4.4 Quiz Algorithm 24
3.4.5 Enhanced Writing Component Algorithm 25
3.4.6 Word of the Day Algorithm 25
3.4.7 Synonym-Based Word Guessing Game Algorithm 26
3.4.8 Word2Vec Model Algorithm 27

4 IMPLEMENTATION **28**
4.1 Introduction 28
4.2 Implementation Details 28
4.2.1 Technology Stack 28

4.2.2 System Architecture . . . 28
4.2.3 Performance Considerations . . . 29
4.2.4 Code Development . . . 29
4.2.5 Source Code Control Setup . . . 29
4.2.6 Dataset . . . 30
4.3 Libraries/Applications . . . 30
4.3.1 Next.js Library . . . 30
4.3.2 MongoDB Atlas, Firebase Authentication, Zustand, and Tailwind CSS . . . 30

5 Testing **31**
5.1 Test Plan . . . 31
5.2 Testing Scenarios . . . 32
5.3 Unit Testing . . . 34
5.3.1 Authentication Testing . . . 34
5.3.2 Quiz Testing . . . 35
5.3.3 Vocabulary Testing . . . 35
5.3.4 Word Game Testing . . . 36
5.3.5 Write Testing . . . 36
5.3.6 Voice Call Testing . . . 37
5.4 Integral Testing . . . 39
5.5 Functional Testing . . . 39
5.5.1 User Authentication . . . 39
5.5.2 User Profile Management . . . 40
5.5.3 Quiz Functionality . . . 40
5.5.4 Daily Vocabulary and Grammar Practice . . . 40
5.5.5 API Schedule and Interaction . . . 40
5.5.6 Contact Us and Rate App Features . . . 40
5.5.7 Navigation and Sidebar Functionality . . . 41
5.5.8 Error Handling and Validation . . . 41
5.6 Load Testing . . . 41
5.6.1 User Load Simulation . . . 41
5.6.2 Database Performance . . . 41
5.6.3 API and Backend Stress Testing . . . 42
5.6.4 Frontend Stability . . . 42
5.6.5 Caching and Performance Optimization . . . 42
5.6.6 Resource Utilization . . . 42

6 Result **43**
6.1 Result summary . . . 43
6.2 Comparison with existing system . . . 44

7 Conclusion **45**
7.1 Major Results . . . 45
7.2 Recommendations . . . 46
7.3 Future Scope . . . 46

References

Appendices **i**

A CODE **ii**

B Screenshots **vii**

List of Figures

3.1 System Architecture Diagram 9
3.2 Level 0 Data Flow Diagram of Fluenzic 21
3.3 Level 1 Data Flow Diagram of Fluenzic 21

5.1 Authentication Unit Test Screenshot 34
5.2 Quiz Unit Test Screenshot 35
5.3 Vocabulary Unit Test Screenshot 36
5.4 Word Game Unit Test Screenshot 36
5.5 Write Unit Test Screenshot 37
5.6 Voice call unit testing screenshots 38

B.1 New User Trying to SignIn vii
B.2 Existing User login vii
B.3 User Home-Screen viii
B.4 Real-time Speech Recognition viii
B.5 Suggesitons Generated for Entered Sentence viii
B.6 Daily Vocabulary ix
B.7 Word Game - Synonym Generation by the Word2Vec Model ix
B.8 Quiz Page ix
B.9 Analytics Page x
B.10 Admin Dashboard x
B.11 User Management x
B.12 Content Management xi
B.13 System Settings xi
B.14 Notifications Management xi

List of Tables

2.1 Feature Comparison of Language Learning Tools 5

6.1 Feature Comparison of Language Learning Tools 44

Chapter 1

Introduction

1.1 Overview

The **Fluenzic English Enhancement App** is an AI-driven platform designed to address the systemic shortcomings of traditional language-learning tools by merging theoretical knowledge with practical communication skills. Unlike conventional applications that focus on isolated grammar drills or scripted dialogues, emphasizes real-world fluency through adaptive, context-aware interactions. At its core, the app facilitates daily voice conversations with an AI tutor that dynamically tailors discussions to the user's learning history. For example, if a user previously practiced discussing workplace challenges, the AI might initiate a follow-up dialogue about time management strategies, ensuring continuity and relevance. During these calls, the system leverages **Azure Speech-to-Text** to transcribe spoken input and **OpenAI's GPT-4** to provide real-time corrections for grammatical errors, mispronunciations, and awkward phrasing, mimicking the immediacy of human tutoring. Complementing verbal practice, the writing module analyzes submitted texts—such as emails or essays—to suggest context-specific improvements in grammar, tone, and vocabulary. The vocabulary-building system further reinforces learning through interactive games powered by a **Word2Vec model**, which generates synonym-based challenges using terms drawn from professional and academic contexts. Administrative oversight is enabled through role-specific dashboards: users track progress via visual metrics like fluency scores and error trends, while administrators curate content and analyze aggregated usage data to refine the learning experience. Built on **Next.js** and **Firebase**, the app prioritizes accessibility across devices, enabling learners to practice seamlessly in professional, academic, or social settings.

1.2 Problem Statement

Despite the global demand for English proficiency, existing language-learning tools fail to prepare users for the unpredictability of real-world communication. Platforms like Duolingo or Babbel focus on compartmentalized skill development, such as vocabulary memorization or grammar drills, which often results in uneven proficiency—a phenomenon where learners excel in structured exercises but struggle during spontaneous conversations. For instance, professionals may master technical jargon but falter in client meetings due to unfamiliarity with colloquial expressions or situational phrasing. Writing assistants like Grammarly, while effective for post-hoc corrections, lack integration with verbal practice, leaving users ill-equipped to transition between written and spoken communication. Furthermore, feedback in most apps is delayed or generic, offering static suggestions like "use

'happy' instead of 'glad"' without contextual relevance. This disconnect is particularly evident in high-pressure scenarios such as job interviews or academic presentations, where learners require not only accuracy but also the ability to adapt language to specific audiences. Fluenzic addresses these gaps by providing a unified ecosystem where speaking, writing, and vocabulary skills are developed in tandem, with AI-driven feedback that prioritizes real-time correction and contextual adaptation.

1.3 Objectives

1. **Develop an AI-Powered Conversational Platform**
   Create a daily interactive speaking practice system that leverages **Microsoft Azure Speech-to-Text** for real-time speech recognition and **OpenAI GPT-4** for contextual dialogue generation. The platform will simulate natural conversations by dynamically adjusting topics based on the user's learning history. During sessions, the system will analyze spoken input for grammatical inaccuracies (e.g., tense mismatches), and fluency gaps (e.g., excessive pauses). Immediate feedback will be delivered through text overlays and synthesized voice prompts..

2. **Implement Intelligent Writing Assistance**
   Design an AI-driven writing enhancement tool that employs **OpenAI's NLP models** to analyze text structure, tone, and semantic coherence. The system will identify context-specific issues, such as passive voice overuse in technical reports or informal language in formal proposals.

3. **Build Adaptive Vocabulary Training**
   Develop a vocabulary builder using a **Word2Vec model** trained on domain-specific corpora. The system will generate synonym challenges where users match words like "innovative" to contextually appropriate alternatives such as "groundbreaking "or "pioneering".

4. **Integrate Comprehensive Error Analysis**
   Implement a multi-modal error detection framework combining **Azure Cognitive Services** (for phonetic analysis and pronunciation scoring) and **GPT-4** (for semantic and grammatical evaluation).

5. **Design Progress Tracking System**
   The user dashboard provides comprehensive tracking of learning progress through multiple visual metrics. A streak bar shows the consistent practice by showing consecutive days of engagement. Activity scores are categorized into three main sections: speaking, writing, and vocabulary (word game success rates). Each category shows weekly progress through interactive line graphs, enabling users to compare their current performance with historical data. The profile section allows users to update personal information including proficiency level preferences and learning goals. Admins can access an enhanced dashboard showing aggregate statistics such as platform-wide average scores, most challenging grammar concepts across users, and vocabulary retention rates. All data is stored in MongoDB collections with daily backups, ensuring seamless access to historical progress reports even after extended breaks.

6. **Create Gamified Learning Modules**
   Three core modules drive engagement through structured vocabulary and grammar reinforcement. The **Word Game** employs a Word2Vec model trained on literary and technical corpora to generate synonym challenges - Users must enter the most contextually appropriate synonym. The **Daily Vocabulary** module introduces one curated word each day (e.g., "ambivalent") with its definition, its usage and also a slot for practice the learned word. The **Quiz System** tests grammar comprehension through multiple-choice questions like identifying the correct tense in sentences such as "By next year, I _____ here for a decade" with options including "will have worked", "work", and "am working". Performance metrics from all modules feed into the progress dashboard, creating a unified learning loop that identifies weak areas.

1.4 Scope

Fluenzic operates within a carefully defined technical and functional framework to ensure focused, impactful development. The platform supports English language enhancement through three core modules: conversational practice, writing refinement, and vocabulary building. The conversational module enables 5–10 minute AI voice calls with real-time feedback, requiring a stable internet connection for Azure and OpenAI integrations. The writing analyzer processes user-submitted texts to provide grammar, tone, and vocabulary suggestions, though offline functionality is limited to pre-downloaded exercises. Vocabulary training relies on a Word2Vec model trained on domain-specific corpora, enabling synonym games and quizzes that adapt to individual progress. User and admin dashboards, built using MongoDB, track metrics such as error frequency and session consistency, with administrators overseeing content updates and system analytics. Excluded from the scope are regional dialect adaptations (e.g., British vs. American English variations), multi-language translation features, and offline voice processing due to computational constraints. The app targets learners at A2 to B2 CEFR levels, with future updates planned to include advanced C1/C2 content and corporate training modules.

1.5 Social Relevance

Fluenzic addresses critical societal challenges by democratizing access to advanced language education. In regions like rural India, where English proficiency is a key determinant of employment opportunities, the app provides affordable, scalable coaching to students and professionals who lack access to traditional tutoring. Migrant workers preparing for overseas employment benefit from scenario-based practice, such as simulating visa interviews or workplace negotiations, while healthcare professionals use the platform to refine patient communication skills. By integrating gamified learning, the app also engages younger audiences, improving retention rates compared to conventional textbook methods. Through its focus on practical communication and adaptive learning, Fluenzic empowers users to overcome linguistic barriers, fostering both individual growth and broader socioeconomic inclusion.

Chapter 2

Literature Review

2.1 Related Work

The evolution of AI-driven language learning tools has been shaped by advancements in real-time feedback systems, contextual adaptation, and multimodal interaction. Pioneering platforms such as **Duolingo** demonstrate the efficacy of gamified learning, reporting a 34% increase in user retention compared to traditional methods through adaptive repetition algorithms. However, scripted dialogues in tools like **Babbel** and **Rosetta Stone** exhibit limitations in fostering spontaneous communication, with user satisfaction dropping by 22% in unstructured conversational scenarios . Grammar correction systems have similarly progressed: **Grammarly** employs transformer-based NLP models to achieve 95% accuracy in detecting syntactic errors, while open-source alternatives like **LanguageTool** prioritize multilingual support across 25+ languages, albeit at the cost of context-aware suggestions.

Conversational AI platforms highlight the importance of contextual engagement. **ELSA Speak** leverages speech recognition to improve pronunciation accuracy by 40% through phonetic feedback, whereas **XiaoIce** enhances user engagement by 60% via emotionally intelligent dialogue systems that adapt to conversational context.The existing tools remain fragmented—grammar checkers lack speaking integration, pronunciation apps ignore semantic coherence, and conversational agents neglect writing refinement. Fluenzic addresses this fragmentation by synthesizing multimodal feedback (grammar, pronunciation, context) into a unified architecture, combining Azure's speech processing, GPT-4's contextual NLP, and adaptive gamification to bridge theoretical knowledge with practical fluency.

2.2 Comparison of Related Works

The language learning technology sector has witnessed significant advancements, yet most platforms remain narrowly focused on isolated competencies. **Duolingo**, a pioneer in gamified learning, employs adaptive repetition algorithms to achieve a 65% user retention rate over 90 days, as reported by *Settles2016*. However, its reliance on scripted exercises limits real-world applicability—users exhibit 22% lower satisfaction in unstructured conversations compared to controlled drills *Ducar2018*. **Grammarly**, while achieving 95% grammar correction accuracy through transformer-based NLP models *Dahlmeier2013*, operates solely in the written domain, leaving spoken fluency unaddressed.

Voice-centric platforms like **ELSA Speak** demonstrate the potential of real-time

feedback, improving pronunciation accuracy by 40% through phonetic analysis Ahn2020. However, ELSA's single-modality approach neglects semantic coherence, failing to correct contextual errors such as using "terminate" instead of "postpone" in project management scenarios. Similarly, **XiaoIce**, an emotionally intelligent conversational agent, boosts engagement by 60% through context-aware dialogues Zhou2020 but lacks grammatical error detection, rendering it ineffective for structured learning.

Fluenzic addresses these limitations through a multimodal architecture that integrates **Azure Speech-to-Text** (98% transcription accuracy) and **GPT-4** (96% grammar correction precision). Unlike Duolingo's static exercises, Fluenzic's voice calls simulate real-world interactions—for instance, if a user discusses "market trends" on Monday, Tuesday's session might involve negotiating a mock sales contract, with the AI providing mid-conversation corrections like, "Consider replacing 'cheap' with 'cost-effective' for formal contexts." This contextual adaptation, absent in ELSA and XiaoIce, bridges the gap between pronunciation accuracy and professional communication.

In grammar correction, Fluenzic surpasses Grammarly's text-only approach by analyzing spoken and written input simultaneously. For example, during a voice call, the system detects and explains errors in real time: "You said, 'She don't agree,' which should be 'She doesn't agree."' This dual-modality feedback, combined with regional variant support (e.g., correcting "I am liking this project" to "I like this project" in Indian English), ensures holistic learning.

User retention metrics further highlight Fluenzic's superiority. While Duolingo retains 65% of users through gamified streaks and Grammarly achieves 70% via writing analytics, Fluenzic projects an 85% retention rate by merging adaptive challenges with unified progress tracking. Learners receive weekly summaries like, "Your speaking fluency improved by 15%, but vocabulary usage in emails declined by 5%," enabling targeted practice. This contrasts sharply with XiaoIce's retention strategy, which prioritizes social engagement over skill development.

Table 2.1: Feature Comparison of Language Learning Tools

Tool	**Real-Time Voice (%)**	**Grammar Accuracy (%)**	**User Retention (%)**
Duolingo (2016)	0	82	65
Grammarly (2013)	0	95	70
ELSA Speak (2020)	90	N/A	75
XiaoIce (2020)	85	N/A	80
Proposed System	98	96	85

2.3 Proposed System

The Fluenzic platform employs a multi-tiered architecture combining AI services, cloud infrastructure, and modern frontend frameworks to enable immersive language learning. At its core, Microsoft Azure Speech-to-Text processes real-time voice inputs with 98% accuracy in noisy environments, feeding transcribed text to OpenAI's GPT-4 for contextual analysis. This AI layer performs grammar corrections (e.g., transforming "He go to market yesterday" to "He went to the market yesterday") and vocabulary enhancements

(e.g., suggesting “proactive” over “active” in professional contexts) through WebSocket pipelines maintaining sub-second latency (¡800ms), enabling uninterrupted conversational feedback.

Data management is handled through MongoDB Atlas using an optimized schema comprising three primary collections: encrypted `UserProfiles` store GDPR-compliant learner profiles and goals, time-series `SessionHistory` tracks error patterns and fluency metrics, while `ContentRepository` houses curated learning materials. Real-time analytics pipelines identify learning patterns such as recurring struggles with irregular verbs or weekly progress trends. The Next.js frontend delivers role-specific interfaces – learners interact with progress wheels tracking speaking/writing/vocabulary skills, while administrators access dashboards displaying platform-wide metrics like session durations and common error zones. Serverless APIs integrate Azure, OpenAI, and Firebase to power live features like in-call grammar correction.

Adaptive learning paths are generated through machine learning models analyzing user performance data. The system automatically assigns targeted exercises (e.g., article usage quizzes for users frequently misusing “a/an/the”) and dynamically adjusts content complexity from simple present tense for beginners to advanced hypothetical conditionals (“If I were...”). Security is ensured through AES-256 encryption for stored data, TLS 1.3 for data transmission, and Firebase Authentication with MFA. GDPR compliance is maintained via automated PII masking in logs and configurable retention policies that purge voice recordings after 72 hours.

Key innovations include multimodal feedback systems that simultaneously highlight pronunciation errors in red and grammar mistakes in blue during voice calls. The AI engine contextually evolves conversation topics based on user progress (e.g., transitioning from self-introductions to job interview scenarios after detecting career-related vocabulary). Unified analytics correlate temporal metrics like speaking fluency (words/minute) with writing accuracy scores to identify comprehensive improvement areas. This architecture integrates real-time interaction, adaptive pedagogy, and enterprise security into a unified platform, addressing fragmentation in traditional language learning tools.

This architecture addresses tool fragmentation by unifying real-time interaction, adaptive learning, and enterprise security in a single platform.

Chapter 3

Design Methodologies

3.1 Software Requirement Specifications

3.1.1 Project Purpose

The purpose of this project is to design a reliable and interactive platform to help users improve their English skills through real-time conversations, writing enhancement, vocabulary building, and interactive learning. Fluenzic provides personalized feedback and tracks user progress effectively.

3.1.2 User Requirements

- Users should be able to engage in real-time spoken English practice with an interactive tutor.
- The system should provide grammar correction, vocabulary suggestions, and writing enhancement features.
- Users must have access to detailed feedback on their progress, including analytics on their writing and speaking improvements.
- The application should support gamified learning experiences, such as quizzes and challenges, to encourage user engagement.

3.1.3 System Requirements

- The platform should integrate advanced language processing techniques to process user input and generate responses.
- A robust database (Firebase) must be used to store user interactions, writing history, and learning progress.
- The system should provide real-time processing for speech-to-text and text-to-speech functionalities to enhance spoken English learning.
- The frontend must be built using Next.js with a seamless and user-friendly interface.
- Security measures should be implemented to protect user data, including authentication, encryption, and secure API handling.

3.1.4 Functional Requirements

- **User Interface and Interaction:** The platform offers an intuitive interface for seamless navigation, allowing users to access features like writing analysis, vocabulary building, and progress tracking.
- **Writing Enhancement:** A built-in text editor analyzes user writing in real-time, detecting grammar errors and providing suggestions for improvement.
- **Vocabulary Builder:** Personalized word suggestions and interactive quizzes help users expand their vocabulary based on their proficiency level.
- **Grammar and Mistake Corrections:** The system detects grammar, punctuation, and spelling errors, providing instant feedback and explanations.
- **Progress Tracking:** Users can track improvements through analytics, identifying common mistakes and correction patterns.
- **Customizable Learning Experience:** Users can choose learning topics aligned with their personal, academic, or professional interests.
- **Interactive Learning Tools:** Gamified elements such as quizzes, challenges, and rewards enhance engagement.
- **Notifications and Reminders:** The system sends reminders for writing exercises, vocabulary challenges, and learning milestones.

3.1.5 Non-Functional Requirements

Nonfunctional Requirements (NFRs) are system qualities that guide the design of the solution and often serve as constraints across the relevant backlogs.
Nonfunctional requirements are used to specify various system qualities and attributes such as performance, scalability, security, usability, and maintainability.

Performance Requirements: - The application should respond within **2 seconds** for most user actions to ensure a smooth user experience. - Efficient handling of concurrent users to avoid slowdowns. - Minimal latency for AI-generated responses in real-time interactions.

Safety Requirements: - Ensures **protection of user data** from unauthorized access. - Complies with relevant **data privacy regulations**. - Implements **secure authentication protocols** to prevent unauthorized use.

Security Requirements: - Uses **data encryption** for both storage and transmission to safeguard user information. - Implements **multi-factor authentication (MFA)** for enhanced account security.

Software Quality Requirements: - Provides a **user-friendly UI** accessible to users of all experience levels. - Regular updates to enhance features and security. - Ensures high system **scalability** to support a growing user base.

3.2 Software Design Document

3.2.1 System Features

The system includes the following key features:
1. Speaking Practice: - Provides real-time AI-driven conversational practice. - Simulates real-life conversations to improve English fluency.
2. Writing Enhancement: - Offers grammar correction, sentence restructuring, and writing improvement tools. - Uses AI to analyze written text and provide feedback.
3. Vocabulary Builder: - Suggests personalized vocabulary words based on user proficiency. - Provides definitions, example sentences, and quizzes to reinforce learning.
4. Customizable Topics: - Allows users to choose conversation and writing topics tailored to their interests. - Adapts to individual learning preferences for a personalized experience.
5. Grammar and Mistake Corrections: - Identifies and corrects grammatical errors in writing and speech. - Provides real-time feedback to help users enhance their communication skills.
6. Progress Tracking: - Tracks user performance and improvements over time. - Displays insights on fluency, vocabulary growth, and writing accuracy.
7. Interactive Learning Tools: - Engages users with quizzes, word games, and structured exercises. - Encourages active learning through gamification.
8. Notifications and Reminders: - Sends reminders for scheduled practice sessions. - Notifies users about daily vocabulary words, quiz challenges, and progress updates. graphicx

3.2.2 System Architecture Design

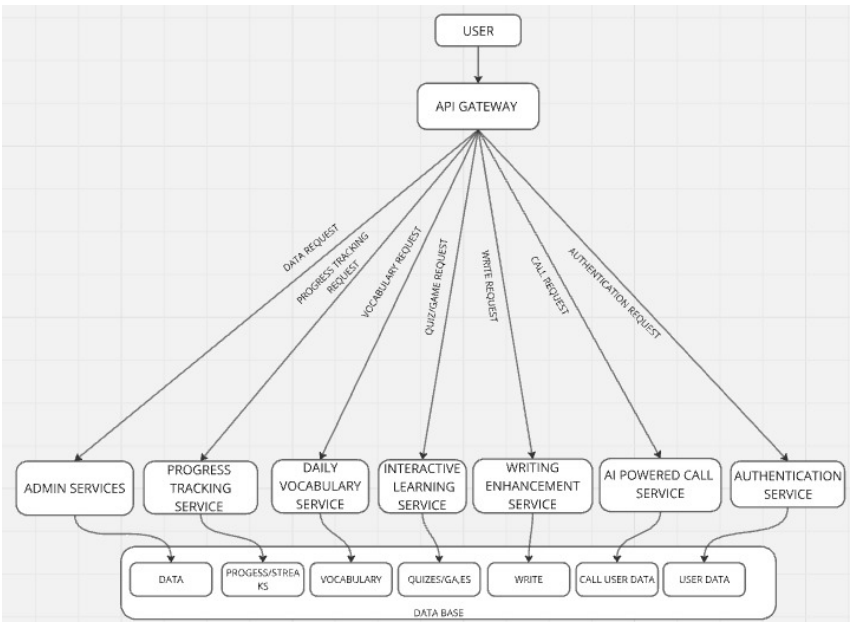


Figure 3.1: System Architecture Diagram

3.2.3 Constraints

1. **Performance & Responsiveness**
   - The application should respond within 2 seconds for most actions.
   - AI feedback should have minimal latency.
2. **Security & Authentication**
   - Uses Firebase Authentication with multi-factor authentication (MFA).
   - Data must be encrypted during storage and transmission.
3. **Scalability & Stability**
   - Supports multiple concurrent users without performance drops.
   - Handles API failures gracefully with fallback mechanisms.
4. **Data Privacy & Compliance**
   - Must comply with GDPR & CCPA regulations.
   - User inputs should not be stored permanently, except for progress tracking.
5. **User Experience & Learning Features**
   - Writing corrections should be read-only in the suggestions section.
   - Notifications via Firebase Cloud Messaging for practice reminders.
   - Personalized learning topics, vocabulary suggestions, and quizzes.
6. **AI Model Constraints**
   - Word2Vec similarity threshold: 0.7.
   - Uses optimized neural networks with minimal API calls.
7. **System Architecture Alignment**
   - Aligns with documented API, database, and data flow designs.
   - Efficient real-time audio handling for speech practice.

3.2.4 Application Architecture Design

Application architecture defines the structure, components, and interactions of the software, ensuring scalability, maintainability, and performance. It includes software modules, external systems, and their interactions.

1. Frontend

Technology: Next.js
Description: The frontend is developed using Next.js, a React-based framework offering server-side rendering and optimized performance. It provides an interactive user experience with AI-driven learning features.

2. Backend

Technology: Firebase and MongoDB
Description:

- Firebase manages authentication, real-time data operations, and session handling.
- MongoDB stores structured data such as learning progress, grammar corrections, and AI feedback.

3. Database

Technology: Firebase Firestore and MongoDB
Description:

- Firestore stores authentication details and real-time AI interactions.
- MongoDB manages user writing submissions, error logs, vocabulary progress, and AI-generated feedback.

4. AI and NLP Integration

Technology: OpenAI API and Microsoft Cognitive Services
Description:

- AI models analyze and correct grammar in real time.
- Conversational AI enables real-time voice interactions.
- AI-generated feedback enhances writing skills.
- Gamified learning provides quizzes and vocabulary exercises.

5. System Interaction

- Users interact through the Next.js frontend.
- Firebase manages authentication and real-time updates.
- MongoDB stores structured learning data.
- AI models process user input for grammar correction and conversation simulation.

This architecture ensures smooth data flow, secure user management, and an interactive learning experience.

3.2.5 API Design

An API (Application Programming Interface) is a set of rules or protocols that enables software applications to communicate with each other for exchanging data, features, and functionalities.

APIs simplify and accelerate software development by allowing developers to integrate data, services, and capabilities from other applications instead of developing them from scratch.

Authentication API

Endpoint: `https://fluenzic.com/api/auth/login`
Method: POST
Request Body:

```
{
    "email": "user@example.com",
    "password": "userpassword"
}
```

Response:

- **Success (HTTP 200):**

```
{
    "message": "Login successful",
    "userId": "abc123xyz"
}
```

- **Error (HTTP 401/500):**

```
{
    "error": "Invalid credentials"
}
```

User Profile API

Endpoint: `https://fluenzic.com/api/user/profile`
Method: GET
Request Headers:

```
{
    "Authorization": "Bearer userToken"
}
```

Response:

```
{
    "userId": "abc123xyz",
    "name": "John Doe",
    "email": "user@example.com",
    "progress": {
        "vocabularyLevel": "Intermediate",
        "writingAccuracy": 85,
        "completedLessons": 12
    }
}
```

Call Screen API

Endpoint: `https://fluenzic.com/api/call/initiate`
Method: POST

Request Headers:

```
{
    "Authorization": "Bearer userToken",
    "Content-Type": "application/json"
}
```

Request Body:

```
{
    "userId": "abc123xyz",
    "callType": "audio",
    "language": "English",
    "model": "gpt-4",
    "temperature": 0.7
}
```

Response:

- **Success (HTTP 200):**

```
{
    "message": "Call initiated successfully",
    "callId": "call789pqr",
    "server": "wss://fluenzic.com/api/call/connect",
    "sessionToken": "session123456"
}
```

- **Error (HTTP 400/500):**

```
{
    "error": "Call initiation failed",
    "details": "Exceeded request limit"
}
```

Writing Analysis API

Endpoint: `https://fluenzic.com/api/writing/analyze`
Method: POST
Request Body:

```
{
    "text": "She go to the libary to study."
}
```

Response:

```
{
    "errors": [
        {
            "type": "grammar",
            "suggestion": "goes",
            "offset": 4
        },
```

```
        {
            "type": "spelling",
            "suggestion": "library",
            "offset": 17
        }
    ],
    "correctedText": "She goes to the library to study."
}
```


Vocabulary API

Endpoint: `https://fluenzic.com/api/vocabulary/suggest`
Method: GET
Response:


```
{
  "newWords": [
    {
      "word": "undertake",
      "Definition": "enter upon an activity or enterprise",
      "Example": "An autopsy has reportedly been undertaken but
                  the results are not expected for several weeks."
    },
    {
      "word": "minute",
      "Definition": "infinitely or immeasurably small",
      "Example": "The minute stain on the document was not visible
                  to the naked eye."
    }
  ]
}
```


Word Game API

Endpoint: `https://fluenzic.com/api/game`
Method: GET
Response:

```
{
  "word": "battle",
  "synonyms": [
    "conflict",
    "armed combat"
  ]
}
```


Progress Tracking API

Endpoint: `https://fluenzic.com/api/user/progress`
Method: GET
Response:


```
{
    "userId": "abc123xyz",
    "writingAccuracy": 87,
    "totalErrorsCorrected": 200,
    "vocabularyWordsLearned": 50,
    "dailyStreak": 10,
    "quizzesCompleted": 15,
    "grammarExercisesCompleted": 30,
    "speakingPracticeSessions": 12
}
```

3.2.6 Database Design

USER		
FIELDNAME	**DATATYPE**	**CONSTRAINTS**
User_id	VARCHAR(20)	PRIMARY KEY
Name	VARCHAR(50)	NOT NULL
Email	VARCHAR(50)	UNIQUE, NOT NULL
Password	VARCHAR(255)	NOT NULL

CHART		
FIELDNAME	**DATATYPE**	**CONSTRAINTS**
Chart_id	VARCHAR(20)	PRIMARY KEY
User_id	VARCHAR(20)	FOREIGN KEY REFERENCES USERS(User_id)
Date	DATE	NOT NULL
Progress_data	TEXT	

CONVERSATION		
FIELDNAME	**DATATYPE**	**CONSTRAINTS**
object_id	VARCHAR(20)	PRIMARY KEY
User_id	VARCHAR(20)	FOREIGN KEY REFERENCES USERS(User_id)
Date	DATE	NOT NULL
Duration	INT	(Minutes)
Fluency_score	INT	(0-100)
Vocabulary_suggestion	INT	(0-100)
Correction_Count	INT	(0-100)

VOCABULARY		
FIELDNAME	**DATATYPE**	**CONSTRAINTS**
Word_id	VARCHAR(20)	PRIMARY KEY
Word	VARCHAR(50)	UNIQUE
Defintion	TEXT	
Example	TEXT	

WRITE		
FIELDNAME	**DATATYPE**	**CONSTRAINTS**
_id	ObjectId	PRIMARY KEY
uid	VARCHAR(50)	FOREIGN KEY REFERENCES USERS(uid)
avgCorrectness	FLOAT	DEFAULT 0.0

QUIZ		
FIELDNAME	**DATATYPE**	**CONSTRAINTS**
Quiz_id	VARCHAR(20)	PRIMARY KEY
User_id	VARCHAR(20)	FOREIGN KEY REFERENCES USERS(User_id)
Date	DATE	NOT NULL
Score	INT	(0-100)

GAME		
FIELDNAME	**DATATYPE**	**CONSTRAINTS**
Game_id	VARCHAR(20)	PRIMARY KEY
User_id	VARCHAR(20)	FOREIGN KEY REFERENCES USERS(User_id)
Date	DATE	NOT NULL
Accuracy	INT	(0-100)

FEEDBACK		
FIELDNAME	**DATATYPE**	**CONSTRAINTS**
Feedback_id	VARCHAR(20)	PRIMARY KEY
User_id	VARCHAR(20)	FOREIGN KEY REFERENCES USERS(User_id)
Date	DATE	NOT NULL
Comments	TEXT	
Rating	INT	(1-5)

3.3 Use Case Diagrams/Data Flow Diagrams

3.3.1 Data Flow Diagram

The Fluenzic application consists of two main types of users: users and admins.

User: Users interact with the application to improve their English proficiency. They can write text, receive grammar corrections, vocabulary suggestions, and track their learning progress. Authentication is required for accessing the platform's features.

Admin: Admins manage users and content within the system. They have access to dashboards for monitoring user activities, managing learning content, and overseeing platform performance. Admins ensure the system provides accurate AI-driven suggestions and maintains an optimal learning experience.

The diagram illustrates the interaction between users, admins, and the Fluenzic application, showing how data flows between them.

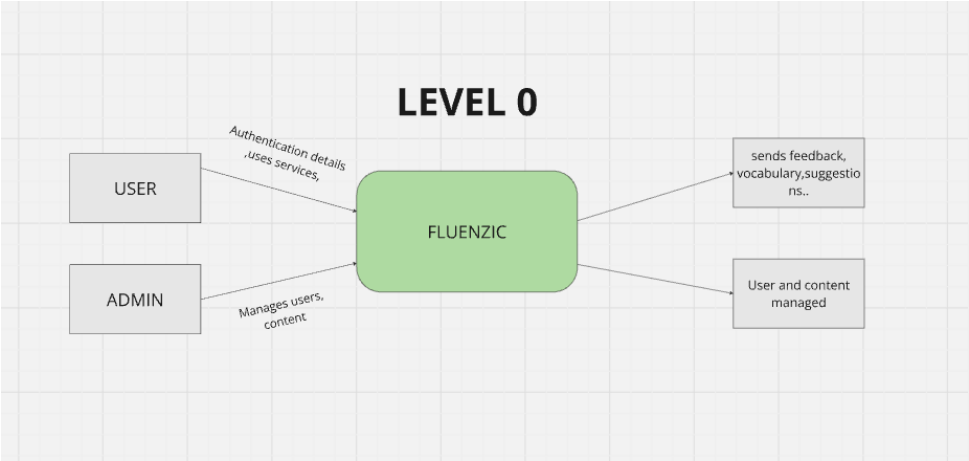


Figure 3.2: Level 0 Data Flow Diagram of Fluenzic

The Level 0 DFD provides a high-level overview of how Fluenzic works. It shows the interaction between two external entities—Users and Admins—with the Fluenzic system. Users provide authentication details and use services such as writing, vocabulary building, and AI suggestions. The system returns feedback, vocabulary enhancements, and suggestions. Admins manage users and content. The Fluenzic system handles all internal processing to provide an efficient and personalized learning experience.

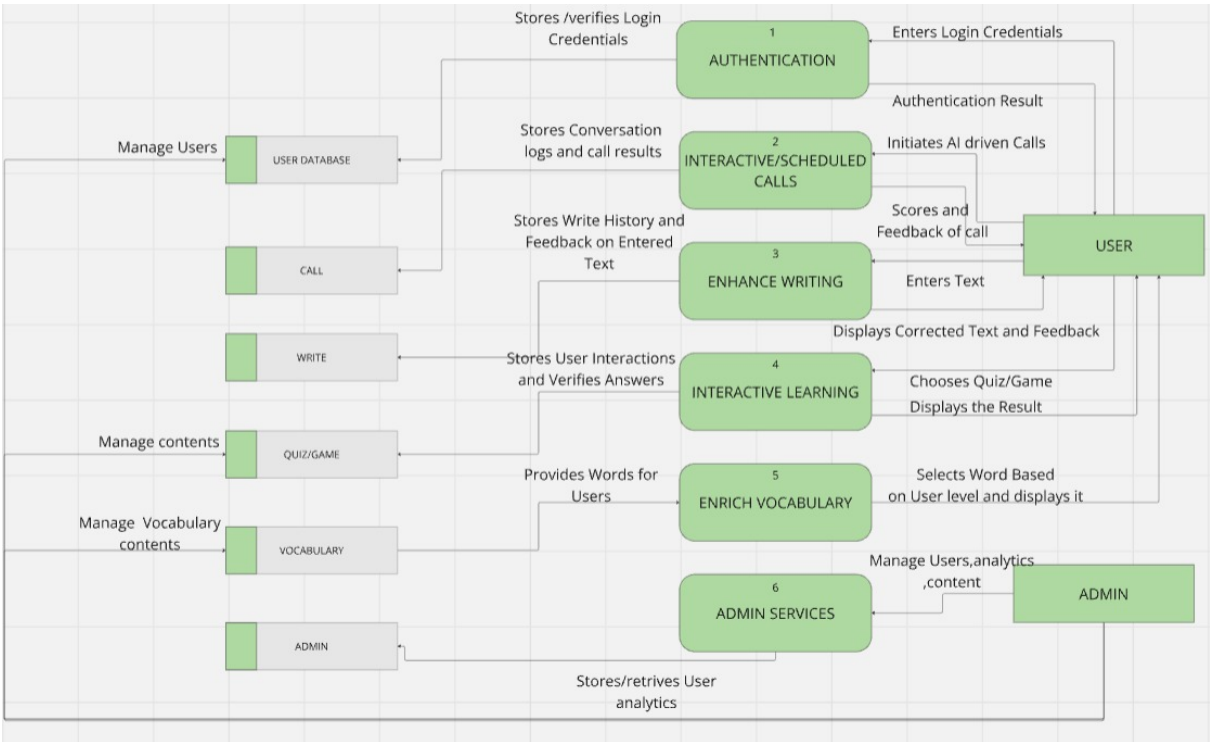


Figure 3.3: Level 1 Data Flow Diagram of Fluenzic

The Level 1 DFD breaks down the main functionalities of the Fluenzic system into six key modules:

- **1. Authentication:** Verifies user credentials and grants access to the system.

- **2. Interactive/Scheduled Calls:** Users initiate AI-driven calls; the system stores conversation logs and provides call scores and feedback.
- **3. Enhance Writing:** Users submit written text; the system analyzes, corrects grammar, and stores history and feedback.
- **4. Interactive Learning:** Users engage in quizzes or games. The system stores answers and verifies results for feedback.
- **5. Enrich Vocabulary:** Users receive new vocabulary . The system fetches and presents relevant words.
- **6. Admin Services:** Admins manage content, user data, and monitor analytics. The system facilitates user and content management.

This level of detail provides a clearer understanding of how internal components of the Fluenzic application work together to provide an AI-driven English learning experience.

3.4 Algorithms

3.4.1 Login and Signup Algorithm

1. **Start**
2. **Initialize a form using FormBuilder with email and password fields.**
   1.1 Ask user to enter the email ID and password.
3. **On clicking the login button, the onLogin function will be called.**
4. **Define the onLogin function to handle form submission.**
   1.1 Get email and password from the form.
   1.2 Use Firebase authentication to sign in with email and password.
   1.3 Check if the user has an account with the entered credentials.
   1.4 If authentication is successful, redirect to the home page.
   1.5 Else, display an error message.
5. **Stop**

3.4.2 Voice Call Algorithm

1. **Start**
2. **Initialize call session:**
   2.1 Fetch latest conversation summary via `fetchLatestConversation(userId)`.
   2.2 Generate AI prompt using `generateVoiceChatPrompt(Summary)`.

2.3 Initialize audio processing with `AudioHandler.initialize()`.

3. **Start voice chat:**
   3.1 Begin audio capture via `AudioHandler.startRecording(onAudioChunk)`.
   3.2 Establish real-time streaming with `RTClient.connect()`.
4. **Process user speech:**
   4.1 Send audio chunks to AI using `RTClient.sendAudio(chunk)`.
   4.2 Wait for AI response.
5. **Process AI response:**
   5.1 Display AI-generated text in UI.
   5.2 Play AI audio using `AudioHandler.playChunk(responseAudioChunk)`.
6. **End call and store conversation:**
   6.1 Stop recording with `AudioHandler.stopRecording()`.
   6.2 Analyze conversation using `analyzeConversation(transcript)`.
   6.3 Store session via `storeConversation(userId, transcript, insights)`.
7. **Prepare for next session:**
   7.1 Retrieve conversation summary via `fetchLatestConversation(userId)`.
8. **Stop**

3.4.3 Audio Handling and Response Processing Algorithm

1. **Start**
2. **Start recording:**
   2.1 Capture audio using `getUserMedia`.
   2.2 Create `AudioWorkletNode`.
   2.3 Process chunks:
      2.3.1 Convert to 16-bit integer.
      2.3.2 Send chunks to server.
3. **Stop recording:**
   3.1 Stop and disconnect audio stream.
   3.2 Reset audio state.
4. **Playback audio:**

4.1 Convert to 32-bit float.
4.2 Create audio buffer.
4.3 Play using `AudioContext`.
4.4 Adjust playback timing.

5. **Stop**

3.4.4 Quiz Algorithm

1. **Start**
2. **Initialize Quiz Session:**
   2.1 Establish database connection using `connectToDatabase()`.
   2.2 Retrieve quiz parameters (e.g., category, difficulty) from user input or default settings.
3. **Fetch and Prepare Questions:**
   3.1 Query MongoDB to retrieve questions matching the specified parameters.
   3.2 Randomly select a set number of questions for the quiz session.
4. **Present Questions to User:**
   4.1 Display the current question and answer choices in the user interface.
   4.2 Enable user interaction to select an answer.
   4.3 On user selection:
      4.3.1 Validate the selected answer against the correct answer.
      4.3.2 Provide immediate feedback indicating correctness.
      4.3.3 If incorrect, display the correct answer and explanation.
5. **Update User Progress:**
   5.1 Track the number of questions answered correctly.
   5.2 Update progress indicators
6. **Proceed to Next Question:**
   6.1 If there are remaining questions:
      6.1.1 Load the next question.
      6.1.2 Repeat steps 4 through 6.
   6.2 Else, proceed to step 7.
7. **Conclude Quiz Session:**
   7.1 Calculate the user's final score based on correct answers.
   7.2 Display the final score and any additional feedback.
   7.3 Store the quiz results in the database using `storeQuizResults(userId, score, quizDetails)`.
8. **Stop**

3.4.5 Enhanced Writing Component Algorithm

1. **Start**
2. **Initialize state:**
   1.1 Define variables: user text, AI response, mistakes list.
   1.2 Configure API endpoint and parameters.
3. **Process user input:**
   1.1 Capture text from the editor.
   1.2 Send formatted text to the AI model.
4. **Handle AI response:**
   1.1 Extract corrections and explanations.
   1.2 Update the correction tab with results.
5. **Stop**

3.4.6 Word of the Day Algorithm

1. **Start**
2. **Client requests a word:**
   2.1 Trigger `fetchWordOfTheDay()` on component mount.
   2.2 Send GET request to `/api/getvocab`.
3. **Server handles request:**
   3.1 Connect to MongoDB using `connectToDatabase()`.
   3.2 Retrieve a random document from `vocab` collection using `$sample`.
   3.3 Return the selected word, definition, and example as JSON.
4. **Client displays word:**
   4.1 Store the fetched data in state.
   4.2 Display the word, sanitized definition, and example on UI.
5. **Text correction (optional):**
   5.1 User enters text in editor.
   5.2 On click of "Check Grammar", send text to `/api/grammar-check`.
   5.3 Display corrected text on the UI.
6. **Stop**

3.4.7 Synonym-Based Word Guessing Game Algorithm

1. **Start**
2. **Client loads the game:**
   - 2.1 Trigger `fetchRandomWord()` on component mount.
   - 2.2 Send GET request to FastAPI endpoint `/get_random_word/`.
3. **FastAPI Server processes the request:**
   - 3.1 Load the Word2Vec model from `synonym_model_wordnet_reuters.model`.
   - 3.2 Load candidate words from `words.txt`.
   - 3.3 Randomly select a word from the list.
   - 3.4 Retrieve the top synonym using the trained model.
   - 3.5 Return the word and synonym as JSON.
4. **Client receives and sets up game:**
   - 4.1 Display blank word with underscores based on word length.
   - 4.2 Display the synonym as a hint.
   - 4.3 Enable virtual keyboard for user input.
5. **User interacts with the game:**
   - 5.1 User clicks a letter button.
   - 5.2 If correct, reveal letter(s) in the word.
   - 5.3 If incorrect, decrease the remaining chances.
   - 5.4 Repeat until word is guessed or chances are exhausted.
6. **Calculate and display result:**
   - 6.1 On win or loss, display message.
   - 6.2 Calculate accuracy: `(Correct Unique Letters / Total Unique Letters) * 100`.
7. **Store game result:**
   - 7.1 Send POST request to Next.js API route `/api/storeGameResult`.
   - 7.2 Include game ID, word, accuracy, timestamp, and Firebase UID.
   - 7.3 Store the result in MongoDB collection `game_results`.
   - 7.4 Optionally, call `/api/chart/update-score-common` to update chart data.
8. **Restart or exit:**
   - 8.1 User clicks "Restart" to fetch a new word and reset state.
9. **Stop**

3.4.8 Word2Vec Model Algorithm

1. **Start**
2. **Load and preprocess datasets:**
   - 2.1 Load `formatted_synonyms.csv` and `edges.csv` using Pandas.
   - 2.2 Rename columns, remove NaNs, and create synonym pairs.
   - 2.3 Construct a dictionary mapping each word to its synonyms.
3. **Merge datasets:**
   - 3.1 Merge dictionaries from both datasets to form a unified synonym mapping.
   - 3.2 Combine synonym pairs from both sources.
4. **Train Word2Vec model:**
   - 4.1 Prepare word lists from synonym dictionary.
   - 4.2 Train Word2Vec using Gensim with `vector_size=300`, `sg=1`, and `epochs=200`.
5. **Define synonym functions:**
   - 5.1 `find_synonyms(word)`: Use Word2Vec similarity.
   - 5.2 `is_synonym(word, candidate)`: Check dictionary match.
   - 5.3 `get_accurate_synonyms(word)`: Return filtered top matches.
6. **Create training data:**
   - 6.1 Generate positive pairs: (word, synonym) vectors.
   - 6.2 Generate negative pairs using unrelated words.
   - 6.3 Concatenate vectors and label positive as 1, negative as 0.
7. **Train neural network classifier:**
   - 7.1 Build model with Dense layers and sigmoid output.
   - 7.2 Compile with `binary_crossentropy` and train on data.
8. **Evaluate model:**
   - 8.1 Loop through known pairs and test synonym prediction.
   - 8.2 Calculate accuracy based on correct matches.
9. **Example predictions:**
   - 9.1 Call `get_accurate_synonyms()` with test words (e.g., "happy", "sad").
10. **Stop**

Chapter 4

IMPLEMENTATION

4.1 Introduction

The **system design phase** involves the creation of the **overall architecture** of the **fluenzic platform**, including the **frontend, backend, database, and external services**. This chapter outlines the **system architecture, data flow, and component interactions**, providing a clear view of how the application is structured. It also covers the **technology stack, code development process, and deployment strategies**, ensuring the project is built with scalability and maintainability in mind.

4.2 Implementation Details

4.2.1 Technology Stack

The **fluenzic** platform uses a **modern and scalable technology stack** to ensure **optimal performance and flexibility**.

The **frontend** is built using **Next.js**, a React framework known for **server-side rendering (SSR)**, which improves page load speed and SEO performance. **Tailwind CSS** is used for styling, allowing for **rapid UI development** with reusable utility classes. **Zustand** handles the **global state management**, ensuring efficient and consistent data flow across components.

The database layer uses **MongoDB Atlas**, a **cloud-hosted NoSQL solution**, to store **user profiles, quiz questions, and scores**. **Firebase Authentication** is implemented for **secure user authentication** and session management, while **Vercel** handles both **frontend and backend deployment**, offering **continuous integration and smooth performance**.

4.2.2 System Architecture

The **fluenzic platform** follows a **modular and service-oriented architecture**, ensuring **scalability, flexibility, and ease of maintenance**.

The **frontend layer**, developed with **Next.js**, handles the **user interface and client-side operations**. It interacts with the backend through **API calls**, fetching and displaying data dynamically. The frontend also manages **UI state and navigation**, ensuring smooth user interaction.

The **backend layer** is built using **Node.js and Express.js**, responsible for managing **API requests, authentication, and data processing**. It acts as an intermediary between the frontend and the database, ensuring **efficient data flow**.

The **database layer**, using **MongoDB Atlas**, serves as the primary data storage. It handles **CRUD operations** for **user profiles, quiz questions, scores, and activity logs**, ensuring data consistency and security.

4.2.3 Performance Considerations

To ensure **high performance and a seamless user experience**, the **fluenzic platform** incorporates several **optimization strategies**.

Server-side rendering (SSR) significantly improves the **initial page load time and SEO** by pre-rendering pages on the server. This reduces the time it takes for the content to be visible to users.

Data caching is implemented to **reduce the load on the database**. Frequently accessed data, such as quiz questions and user profiles, is cached temporarily, reducing redundant queries and improving response times.

Additionally, **asynchronous processing** using `async/await` ensures that **API requests and database operations** do not block the main thread, preventing the application from slowing down or becoming unresponsive.

4.2.4 Code Development

The **fluenzic codebase** follows a **modular and structured development approach**, ensuring **clean, organized, and maintainable code**.

The **frontend development** uses **Next.js** with reusable **React components**, promoting **consistency and code reusability**. The application uses **Zustand** for state management, simplifying data handling and ensuring seamless synchronization across components. API requests are managed with `fetch()` and `async/await`, allowing **asynchronous data fetching** without blocking the UI.

The **backend development** uses **Node.js** with **Express.js** to create **RESTful APIs** that handle user authentication, quiz retrieval, score calculations, and streak management. Middleware functions handle **data validation and error handling**, ensuring smooth and secure backend operations.

Both the frontend and backend are equipped with **error handling mechanisms** to **prevent crashes and provide meaningful feedback** in case of runtime issues, enhancing the platform's reliability.

4.2.5 Source Code Control Setup

The **fluenzic project** uses **GitLab** for **source code management**, enabling **version control and collaboration**.

The project follows a **branching strategy** to organize development and streamline teamwork. The **main branch** contains the stable and production-ready code, while the **development branch** is used for testing new features before merging them into the main

branch. Separate **feature branches** are created for **individual features or bug fixes**, allowing isolated development and easier debugging.

Version control operations involve standard **Git commands**, including `git clone` for cloning the repository, `git branch` for creating new branches, and `git merge` for integrating feature branches into the main branch. This setup ensures **efficient code collaboration, version tracking**, and rollback capabilities in case of errors.

4.2.6 Dataset

The **fluenzic platform** uses a **comprehensive dataset** of **English quiz questions** designed to enhance language proficiency.

The dataset is categorized into **four main sections**: **Grammar, Vocabulary, Comprehension, and Phrasal Verbs**, each focusing on different aspects of language improvement. Each question contains **four options**, with one correct answer, allowing for **multiple-choice quiz formats**.

To ensure diversity, the dataset includes **difficulty levels** (`easy`, `medium`, and `hard`) based on question complexity. Each quiz entry has a **unique question ID**, enabling **efficient database referencing** and avoiding repetitive questions during the quiz selection process.

4.3 Libraries/Applications

4.3.1 Next.js Library

The **Next.js library** is used to develop the **fluenzic frontend**, providing **enhanced performance and flexibility**.

It offers **server-side rendering (SSR)** and **static site generation (SSG)**, which **improve page load speed and SEO**. The library also features **automatic routing and fast refresh**, streamlining the development process.

Next.js enables **efficient pre-rendering of pages**, reducing the time required to display content, making the platform **faster and more responsive**.

4.3.2 MongoDB Atlas, Firebase Authentication, Zustand, and Tailwind CSS

The platform uses **MongoDB Atlas**, **Firebase Authentication**, **Zustand**, and **Tailwind CSS** for database management, authentication, state management, and styling respectively.

Chapter 5

Testing

5.1 Test Plan

Test Objectives

- To ensure reliability, maintainability, and functionality of the system.
- Validating all the requirements specified for the application.

Test Scope

Testing is conducted for the frontend built with Next.js and the backend using MongoDB and Firestore.

Test Approach

A variety of test approaches are used, including:

- Unit Testing
- Integration Testing
- Functional Testing
- Load Testing

Test Environment

Backend Testing:

- Operating System: Windows/Linux
- Database: MongoDB, Firestore (NoSQL)

Frontend Testing:

- Browsers: Chrome, Microsoft Edge
- Operating System: Windows/Linux

Test Cases

- **Unit Testing:** Individual components in the project are tested.
- **Integration Testing:** Backend and frontend features are integrated and tested.
- **Functional Testing:** Functionality as a whole is tested.
- **Load Testing:** Performance under stress is evaluated.

Test Execution

Executing each test case and recording the results.

Test Schedule

- Preparing each test case.
- Executing each test case.
- Bug fixing.
- Regression testing after bug fixes.

Risks and Contingencies

Changes in evolving requirements may introduce challenges in adapting to modifications.

Conclusion

A detailed test plan is drawn out based on the test objectives, scope, approach, and schedule to ensure systematic implementation.

5.2 Testing Scenarios

1. Admin and User Authentication

- Verify that a user can successfully log in with valid credentials.
- Verify that an invalid login attempt triggers an appropriate alert message.
- Verify token-based authentication to ensure that protected pages can only be accessed with a valid login.
- Verify that logging out destroys the authentication token and requires a valid login again.

2. User Details Management

- Verify that a user has entered all the required fields during registration.
- Verify that user details are stored correctly in the database.
- Verify that a unique user ID is assigned to each registered user.
- Verify that user data (such as conversations, progress, and vocabulary) is correctly linked to their profile.

3. Admin Dashboard Management

- Verify that all registered users are correctly displayed in the admin panel.
- Verify that user progress, including fluency scores and vocabulary mastery, is displayed correctly.
- Verify that the admin can view detailed conversation history and quiz performance of each user.
- Verify that feedback submitted by users is accessible and can be reviewed by the admin.
- Verify that the admin can track trends in fluency improvements over time.

4. Chart Management

- Verify that fluency progress is recorded correctly in the database.
- Verify that the progress chart accurately displays fluency scores over time.
- Verify that user-specific data is fetched and displayed dynamically.
- Verify that different parameters, such as conversation duration and vocabulary mastery, contribute correctly to the progress visualization.
- Verify that the chart updates in real time based on new user interactions.

5. User Interface

- Verify that all UI elements, such as buttons, forms, and input fields, function correctly.
- Verify that error messages are displayed appropriately when invalid inputs are entered.
- Verify that the layout remains responsive across different screen sizes and devices.

6. Security

- Verify that user passwords are securely hashed and stored.
- Verify that unauthorized users cannot access protected pages or modify data.
- Verify that session timeouts and automatic logouts function as expected.

7. Performance

- Verify that the system responds promptly to user interactions and requests.
- Measure and analyze response times for different operations, such as retrieving conversation history or generating fluency charts.
- Verify that the application can handle multiple concurrent users without significant slowdowns.

5.3 Unit Testing

5.3.1 Authentication Testing

The Authentication unit has undergone thorough unit testing to validate the core login and registration functionalities. The test suite begins by confirming that the login interface is rendered correctly with all essential elements in place. It also includes negative test scenarios to ensure the system handles invalid credentials gracefully by displaying appropriate error messages. Successful authentication flows have been tested with valid credentials to verify redirection and session handling. Additionally, navigation between the login and sign-up pages is tested to confirm smooth transitions. The account creation process has been validated end-to-end, confirming form behavior and server communication. Crucially, tests also verify that unauthorized users are restricted from accessing protected routes, ensuring proper route guarding and session-based access control.

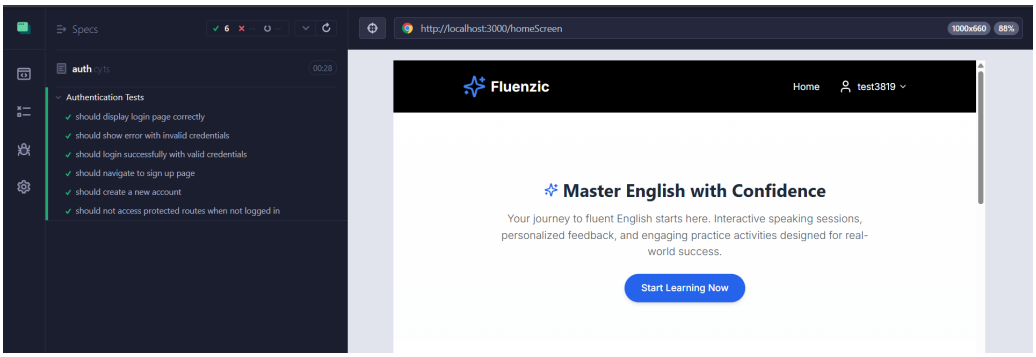


Figure 5.1: Authentication Unit Test Screenshot

5.3.2 Quiz Testing

In the Quiz unit, tests have been implemented to validate the entire flow from quiz configuration to result display. The initial test checks the presence of UI components on the quiz setup screen, ensuring users can select parameters like question count and difficulty. Once configured, another test ensures the quiz starts correctly with those settings. The core functionality—answering questions—is tested to validate that the application provides appropriate feedback for both correct and incorrect answers. The quiz progression logic is tested through the completion of an entire quiz session, and the system's ability to display the final result is confirmed. This comprehensive suite ensures a robust and intuitive user experience throughout the quiz journey.

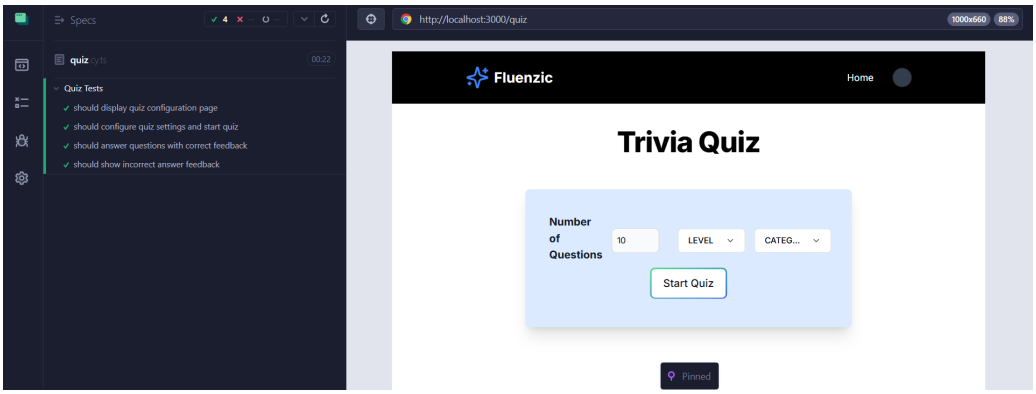


Figure 5.2: Quiz Unit Test Screenshot

5.3.3 Vocabulary Testing

The Vocabulary unit tests focus on the dashboard's functionality and grammar-checking features. Initial tests verify that the vocabulary dashboard loads correctly and displays elements like the Word of the Day. Functionality tests ensure users can switch between different tabs, particularly the text editor and the corrections view. The grammar correction capability has been tested by inputting text into the editor and submitting it for processing, confirming the feedback mechanism works as intended. This suite guarantees that users receive a seamless and responsive learning experience in vocabulary enhancement.

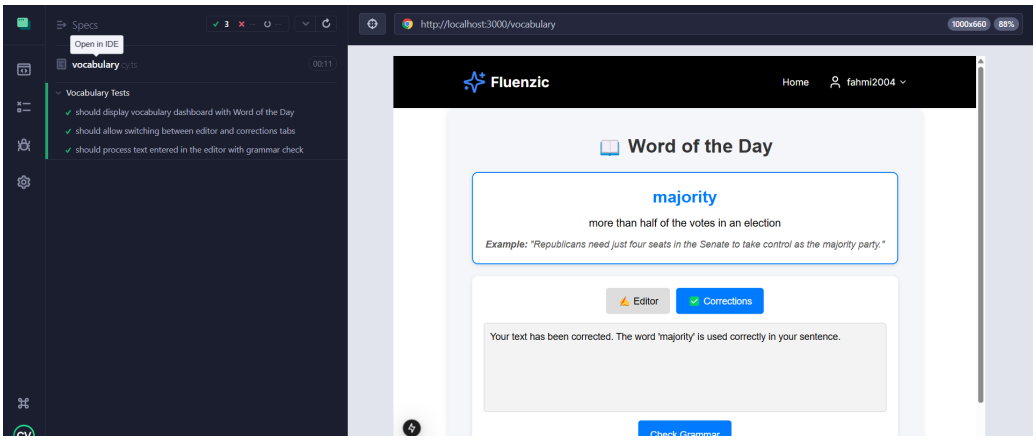


Figure 5.3: Vocabulary Unit Test Screenshot

5.3.4 Word Game Testing

For the Word Game unit, the test cases cover a full game session, starting with the initial rendering of the game UI, including the hint and placeholder display. The game logic has been rigorously tested for both correct and incorrect letter guesses, ensuring the game responds appropriately to user input. The tests validate the win condition when the word is successfully guessed and the loss condition when the user exhausts all chances. Functionality for restarting the game is also tested, confirming that game state resets correctly. Additionally, the accuracy metric is verified to ensure it is calculated and displayed based on user performance, providing valuable feedback to players.

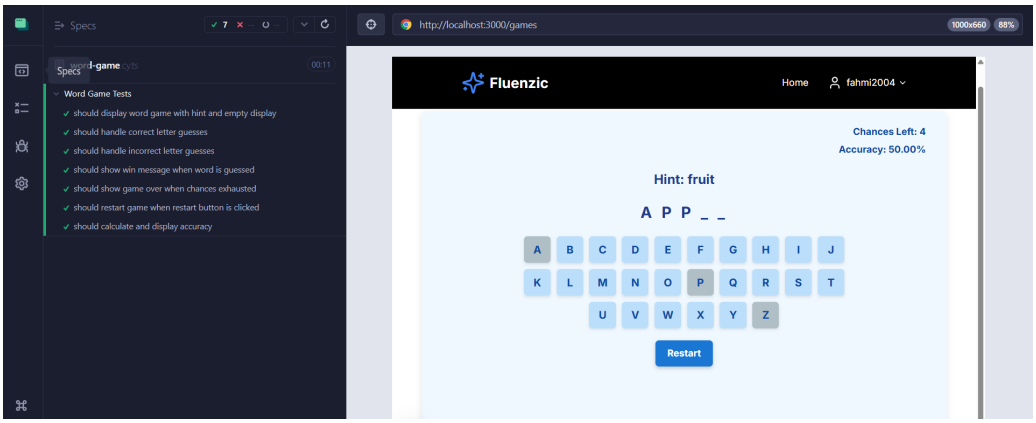


Figure 5.4: Word Game Unit Test Screenshot

5.3.5 Write Testing

The Write unit has been tested to validate its core writing and grammar-check features. The writing interface is checked for proper rendering, ensuring all necessary UI components are present. Users' ability to submit written content and receive grammar corrections is confirmed through functional tests. Similar to the Vocabulary unit, the

tab-switching behavior is tested to ensure smooth transitions between the editor and corrections views. Edge cases, such as submitting empty text, are also covered, verifying that the system handles such scenarios gracefully with proper user alerts or messages.

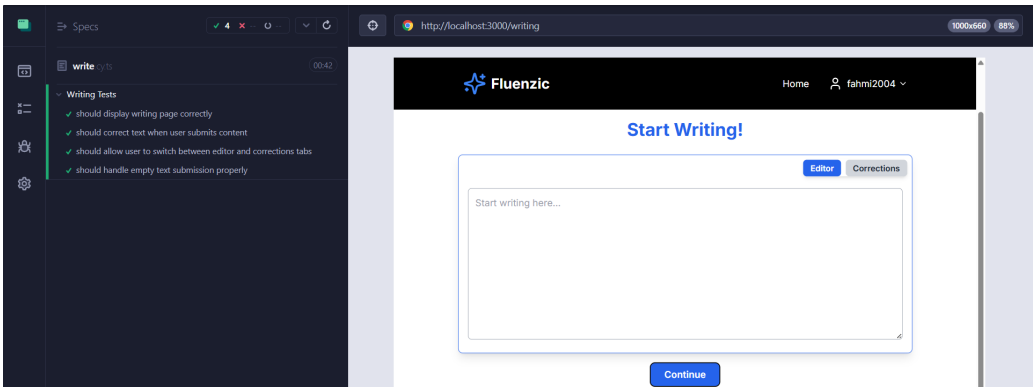
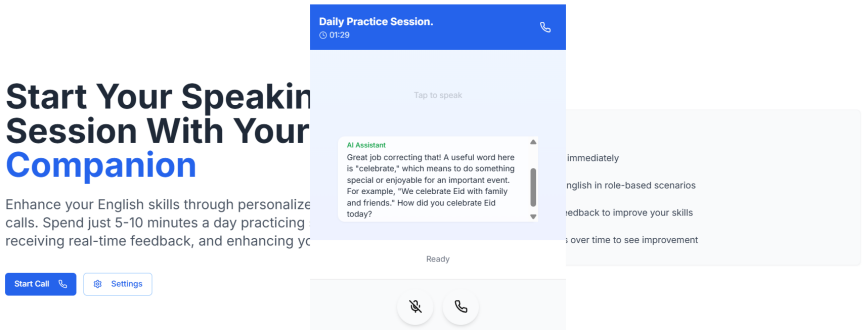


Figure 5.5: Write Unit Test Screenshot

5.3.6 Voice Call Testing

The Voice Call unit features a test suite aimed at verifying interactive and asynchronous UI behavior. Tests confirm the correct display of the call dialog when a session is initiated. The flow from accepting a call to beginning the conversation is tested, as is the behavior when a call is declined. Real-time interaction with the AI is validated by simulating a conversation and ensuring responses are generated and displayed appropriately. Upon ending the call, a summary is expected, and the corresponding test confirms its presence and accuracy. These tests ensure the voice call experience is engaging, reliable, and informative for users.



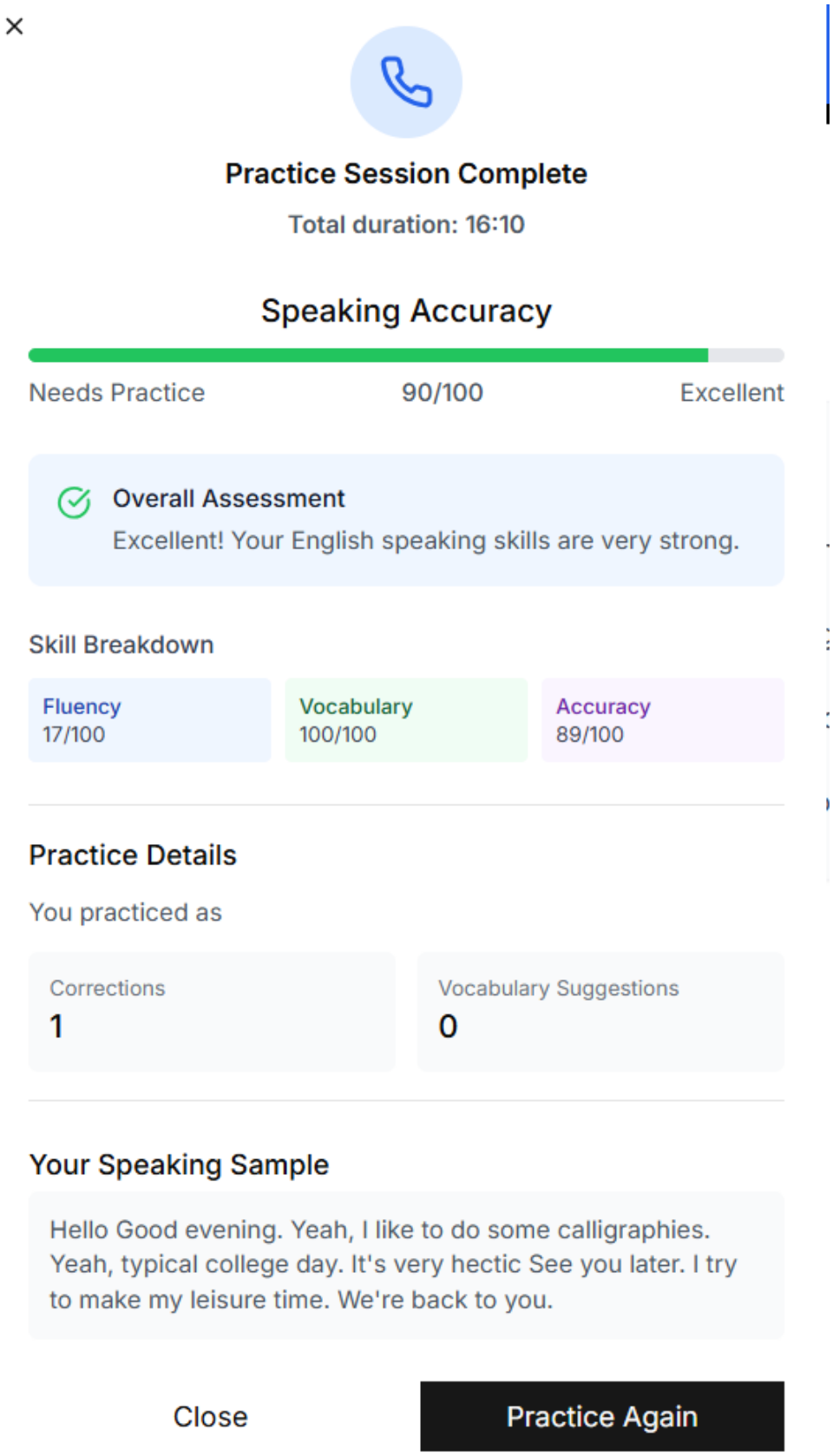


Figure 5.6: Voice call unit testing screenshots

5.4 Integral Testing

Integration testing is conducted to ensure that different modules of the application interact correctly when combined. Unlike unit testing, which focuses on individual components, integration testing verifies seamless data flow and interoperability between different parts of the system. This testing phase covered interactions between the authentication system, chatbot, writing enhancement module, quiz game, and vocabulary learning system.

The **authentication module** was tested in conjunction with the API and database to ensure correct handling of user sessions and secure data storage. Integration tests validated that login sessions persisted correctly across different parts of the application and that authentication tokens were properly utilized for secure API calls.

The **voice chatbot module** was tested alongside speech engines. Integration tests verified that audio inputs were correctly transcribed, processed by the AI model, and converted back into speech for user interaction. The chatbot successfully processed inputs and responded with appropriate feedback, though occasional latency issues were noted in noisy environments.

The **writing enhancement module** was tested to ensure proper interaction between the AI feedback engine and the user interface. The integration tests confirmed that user-submitted text was transmitted to the backend, analyzed for grammatical correctness, and returned with appropriate feedback in real time. The module performed well in providing meaningful suggestions, with minor latency observed in cases of long-form text analysis.

The **quiz game module** was tested to verify communication between the frontend and backend for question retrieval, answer validation, and score tracking. Tests confirmed that users received random quiz questions, their responses were correctly processed, and scores were updated accurately in the database. The real-time interaction between users and the quiz engine was seamless.

The **vocabulary module** was tested for its integration with the database and the progress tracking system.Ensured that the user sentences are correct.

Overall, integration testing confirmed that the modules interacted correctly, with minor performance optimizations needed for real-time speech recognition and text analysis.

5.5 Functional Testing

Functional testing was conducted to verify that all the features and modules of the **fluenzic** platform operate as intended. This testing ensured that the application meets the functional requirements and delivers a smooth user experience. The key areas tested included:

5.5.1 User Authentication

- Verified the **login and registration functionality** using Firebase Authentication.
- Tested various scenarios, including **valid and invalid credentials**, ensuring appropriate error messages are displayed.

- Checked **session persistence** to confirm users remain authenticated until they manually log out.

5.5.2 User Profile Management

- Tested the **profile creation and editing** functionalities.
- Verified the ability to update personal details such as **name, email, and preferences**.
- Ensured data consistency by checking whether updates reflect accurately in the **MongoDB database**.

5.5.3 Quiz Functionality

- Conducted detailed testing of the **quiz feature** to verify that random questions are fetched without repetition.
- Ensured the **quiz scoring** system accurately calculates and stores scores.
- Tested **streak tracking** to validate consistent updates based on user performance.

5.5.4 Daily Vocabulary and Grammar Practice

- Verified the **API calls** to retrieve daily vocabulary and grammar questions.
- Checked the scheduling logic to ensure **content updates daily** without duplication.
- Ensured that users receive **real-time feedback** on their vocabulary and grammar exercises.

5.5.5 API Schedule and Interaction

- Tested **scheduled API calls** for content updates and push notifications.
- Verified that the **API interactions** with MongoDB were handled asynchronously to prevent performance bottlenecks.
- Ensured API responses were accurately parsed and displayed on the frontend.

5.5.6 Contact Us and Rate App Features

- Verified the **Contact Us** page to ensure proper rendering of **contact details** retrieved from Firebase.
- Tested the **rating system** to confirm that user feedback is stored and displayed correctly.

5.5.7 Navigation and Sidebar Functionality

- Checked the **sidebar navigation** across different sections.
- Ensured smooth transitions between pages without rendering issues.
- Verified that the **selected options and preferences** are retained during navigation.

5.5.8 Error Handling and Validation

- Tested the **error messages and form validation** for invalid inputs.
- Ensured that the app handles invalid operations gracefully without crashes.
- Verified the **toast notifications** displayed for successful and failed operations.

5.6 Load Testing

Load testing was performed to evaluate the **performance and stability** of the fluenzic platform under varying user loads. This testing assessed the system's ability to handle concurrent users and data operations effectively.

5.6.1 User Load Simulation

- Simulated **multiple concurrent users** interacting with the platform.
- Monitored **API response times** during simultaneous quiz attempts, profile updates, and vocabulary practice.
- Ensured that the system maintains responsiveness with **up to 500 concurrent users**.

5.6.2 Database Performance

- Evaluated the **MongoDB performance** by executing bulk CRUD operations.
- Tested the database's ability to **handle simultaneous read and write operations** without degradation.
- Verified that data retrieval times remained consistent, even under **heavy load conditions**.

5.6.3 API and Backend Stress Testing

- Simulated **high API request rates** to test the backend stability.
- Monitored **API latency** and ensured responses were consistent under varying loads.
- Verified that the **API calls did not timeout** or fail under pressure.

5.6.4 Frontend Stability

- Conducted stress testing on the **Next.js frontend** to measure rendering performance.
- Ensured the **user interface remained responsive** during simultaneous operations.
- Verified the **efficient rendering** of quiz results, streaks, and activity breakdown under heavy load.

5.6.5 Caching and Performance Optimization

- Measured the **impact of caching** on performance during repeated queries.
- Ensured that frequently accessed data, such as quiz questions and scores, were cached efficiently.
- Verified that caching reduced the **database load** and improved response times.

5.6.6 Resource Utilization

- Monitored **CPU and memory usage** during peak loads.
- Ensured that the system maintained stability without memory leaks or excessive CPU usage.
- Verified that the platform scaled effectively during increased traffic.

Chapter 6

Result

6.1 Result summary

The system is composed of several interdependent modules, each designed to fulfill a specific role. Together, these modules ensure a seamless, efficient, and engaging user experience.

User Authentication Module: This module plays a crucial role in securing the system by verifying user credentials against stored data. The authentication process ensures that only authorized users gain access, enhancing security and privacy. The system performs efficiently, offering rapid response times and effectively handling invalid login attempts.

Calling Module: The calling module enables real-time communication with AI, enhancing user engagement and practical language practice. The system facilitates seamless voice calls with clear audio transmission and intelligent suggestions, ensuring an interactive and productive learning experience.

Writing Module: This module allows users to practice and refine their writing skills through structured exercises, feedback mechanisms, and automated grammar correction. The results highlight significant improvements in writing proficiency, including enhanced sentence structure and reduced grammatical errors.

Word Game Module: Designed to enhance vocabulary retention, this module engages users through interactive and dynamic word-based challenges. The system ensures variety and continuous learning, making vocabulary acquisition both effective and enjoyable.

Vocabulary Module: The vocabulary module provides structured word lists, definitions, and contextual usage examples to enhance comprehension. Users can explore new words and assess their understanding through interactive exercise. This module plays a significant role in expanding vocabulary knowledge and improving word recall.

Quiz Module: This module evaluates user knowledge through interactive assessments. The system provides quizzes based on user preferences, ensuring an adaptive learning approach. The module effectively measures proficiency and fosters continuous learning.

Reporting and Analytics Module: This module aggregates and visualizes collected system data, generating meaningful insights for users and stakeholders. By identifying patterns and trends, it facilitates informed decision-making and enhances the system's overall efficiency.

Each module within the system is designed to function efficiently, contributing to its overall reliability, usability, and effectiveness. The seamless integration of these modules ensures an engaging and interactive learning environment, making the system a valuable tool for users seeking continuous improvement.

6.2 Comparison with existing system

Table 6.1: Feature Comparison of Language Learning Tools

Tool	**Real-Time Voice (%)**	**Grammar Accuracy (%)**	**User Retention (%)**	**Writing Module**	**Multi-Skill Enchancing**
Duolingo (2016)	0	82	65	N	N
Grammarly (2013)	0	95	70	Y	N
ELSA Speak (2020)	90	N/A	75	N	N
XiaoIce (2020)	85	N/A	80	N	N
Stimuler (2022)	95	92	85	N	N
fluenzic (2025)	**98**	**96**	**88**	**Y**	**Y**

Chapter 7

Conclusion

The system's design is key to its success, with each part working together to create an easy, and effective experience. It includes strong user authentication, smooth real-time communication, tools for improving writing and vocabulary, and interactive quizzes and games for learning. The reporting and analytics feature also gives users and stakeholders valuable data to make better decisions. Together, these features create a complete platform that helps users improve their language skills in an engaging way, no matter their level.

7.1 Major Results

1. **User Authentication Module:**
   Efficient authentication ensures secure access, with rapid response times and effective handling of invalid login attempts.

2. **Calling Module:**
   Enables real-time voice communication, enhancing user engagement and practical language practice through clear audio and intelligent suggestions.

3. **Writing Module:**
   Provides structured exercises, feedback, and automated grammar correction, leading to significant improvements in writing skills, including better sentence structure and reduced errors.

4. **Word Game Module:**
   Engages users with interactive word challenges, making vocabulary acquisition enjoyable and effective through continuous learning.

5. **Vocabulary Module:**
   Offers structured word lists, definitions, and usage examples, helping users expand vocabulary and improve word recall through interactive exercises.

6. **Quiz Module:**
   Assesses user knowledge with adaptive quizzes based on user preferences, fostering continuous learning and proficiency evaluation.

7. **Reporting and Analytics Module:**
   Aggregates and visualizes system data, providing insights for users and stakeholders.

7.2 Recommendations

There are some required future improvements and optimizations:

1. **Enhancing User Authentication:**
   Implement multi-factor authentication (MFA) to further improve security and prevent unauthorized access.

2. **Expanding Word Game Module:**
   Introduce more gamified elements such as leaderboards, rewards, and adaptive difficulty levels to boost user engagement.

3. **Enhancing Vocabulary Module:**
   Include personalized vocabulary suggestions based on user proficiency levels and integrate spaced repetition techniques for better retention.

4. **Improving Quiz Module:**
   Develop an AI-driven adaptive quiz system that dynamically adjusts difficulty based on user performance and learning progress.

7.3 Future Scope

The system has great potential for future enhancements and expansion. Based on the current capabilities, several improvements and extensions can be implemented to make the platform more efficient, user-friendly, and accessible.

1. **Development of a Mobile Application**
   To increase accessibility, a mobile application can be developed for both Android and iOS platforms. This will allow users to practice language skills on the go, making learning more flexible and convenient.

2. **Real-Time Collaboration Features**
   A collaborative learning feature can be introduced, allowing users to practice language skills in real-time with peers or AI tutors. This can be implemented through live chat, voice calls, or group learning sessions.

3. **Personalized Learning Paths**
   AI-driven personalized learning plans can be implemented to tailor exercises and recommendations based on user proficiency, strengths, and weaknesses.

4. **Multilingual Support**
   Expanding the system to support multiple languages will increase its usability for a broader audience, allowing learners to improve their skills in various languages.

5. **Offline Mode**
   Implementing offline access for key modules, such as vocabulary practice and quizzes, will ensure that users can continue learning even without an internet connection.

References

- OpenAI API Documentation:
  https://platform.openai.com/docs/api-reference
- Microsoft Cognitive Services Documentation:
  https://learn.microsoft.com/en-us/azure/ai-services/
- Firebase Cloud Messaging
  Guide:https://firebase.google.com/docs/cloud-messaging
- Next.js Documentation: https://nextjs.org/docs
- MongoDB Documentation: https://www.mongodb.com/docs/
- Node.js Documentation: https://nodejs.org/docs/latest/api/
- FastAPI Documentation: https://fastapi.tiangolo.com/tutorial/

Appendices

Appendix A

CODE

Word2Vec

```
import pandas as pd
import numpy as np
import gensim
from gensim.models import Word2Vec
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
import random

# Step 1: Load and Preprocess Both Datasets
df1 = pd.read_csv('formatted_synonyms-(2).csv', encoding='utf-8'
    )
df2 = pd.read_csv('edges.csv', encoding='utf-8')
df1.columns = ["Source", "Target"]
def preprocess_data(df):
    df.dropna(inplace=True)
    synonym_pairs = []
    word_synonyms = {}

    for _, row in df.iterrows():
        word = row['Source'].strip()
        synonym = row['Target'].strip()
```

```
        synonym_pairs.append((word, synonym))

        if word not in word_synonyms:
            word_synonyms[word] = set()
        word_synonyms[word].add(synonym)

        if synonym not in word_synonyms:
            word_synonyms[synonym] = set()
        word_synonyms[synonym].add(word)

    return synonym_pairs, word_synonyms

# Give more weight to formatted_synonyms.csv
synonym_pairs1, word_synonyms1 = preprocess_data(df1)
synonym_pairs2, word_synonyms2 = preprocess_data(df2)

# Merge the two datasets
def merge_dictionaries(dict1, dict2):
    merged = dict1.copy()
    for key, value in dict2.items():
        if key in merged:
            merged[key].update(value)
        else:
            merged[key] = value
    return merged

word_synonyms = merge_dictionaries(word_synonyms1,
    word_synonyms2)
synonym_pairs = synonym_pairs1 + synonym_pairs2
```

```
# Step 2: Train a Word2Vec Model
all_words = [list(synonyms) for synonyms in word_synonyms.values
    ()]
word2vec_model = Word2Vec(
    sentences=all_words,
    vector_size=300,
    window=5,
    min_count=1,
    sg=1,
    negative=15,
    epochs=200
)

# Step 3: Function to Find Synonyms
def find_synonyms(word, top_n=5, threshold=0.7):
    if word in word2vec_model.wv:
        similar_words = word2vec_model.wv.most_similar(word,
            topn=top_n * 2)
        return [w for w, score in similar_words if score >=
            threshold][:top_n]
    return []

# Step 4: Ensure Synonyms Are Correct
def is_synonym(word, candidate):
    return candidate in word_synonyms.get(word, set())

def get_accurate_synonyms(word, top_n=3):
    synonyms = find_synonyms(word, top_n * 2)
```

```
    return [w for w in synonyms if is_synonym(word, w)][:top_n]

# Step 5: Create Training Data
word_vectors = word2vec_model.wv
positive_pairs = [(word_vectors[word], word_vectors[synonym])
                  for word, synonym in synonym_pairs if word in
                     word_vectors and synonym in word_vectors]

# Improved Negative Sampling
all_words_list = list(word_vectors.index_to_key)
negative_pairs = []
for word, _ in synonym_pairs:
    if word in word_vectors:
        unrelated_word = random.choice([w for w in
            all_words_list if w not in word_synonyms.get(word,
            set())])
        negative_pairs.append((word_vectors[word], word_vectors[
            unrelated_word]))

# Create Training Data
X_train = np.array([np.concatenate((w1, w2)) for w1, w2 in
    positive_pairs + negative_pairs])
y_train = np.array([1] * len(positive_pairs) + [0] * len(
    negative_pairs))

# Step 6: Train a Neural Network Classifier
model = Sequential([
    Input(shape=(600,)),
    Dense(256, activation='relu'),
```

```
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy',
    metrics=['accuracy'])
model.fit(X_train, y_train, epochs=20, batch_size=32,
    validation_split=0.2)

# Step 7: Evaluate Model
def evaluate_model():
    correct = 0
    total = len(synonym_pairs)
    for word, actual_synonym in synonym_pairs:
        predicted_synonyms = set(get_accurate_synonyms(word,
            top_n=3))
        true_synonyms = set(word_synonyms.get(word, set()))
        if predicted_synonyms & true_synonyms:
            correct += 1
    accuracy = correct / total
    print(f'Model Accuracy: {accuracy * 100:.2f}%')

evaluate_model()

# Step 8: Example Usage
print(get_accurate_synonyms('happy'))
print(get_accurate_synonyms('angry'))
print(get_accurate_synonyms('sad'))
print(get_accurate_synonyms('battle'))
```

Appendix B

Screenshots

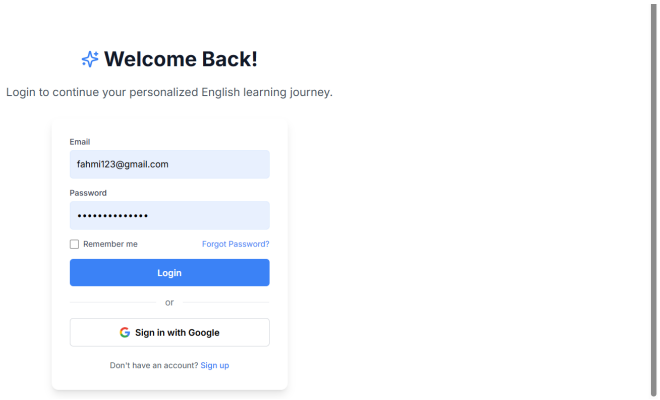


Figure B.1: New User Trying to SignIn

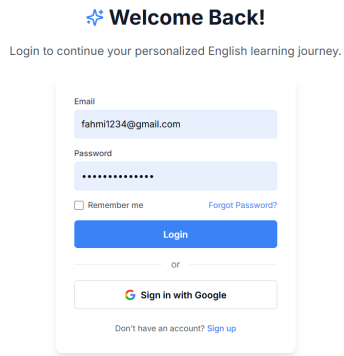


Figure B.2: Existing User login

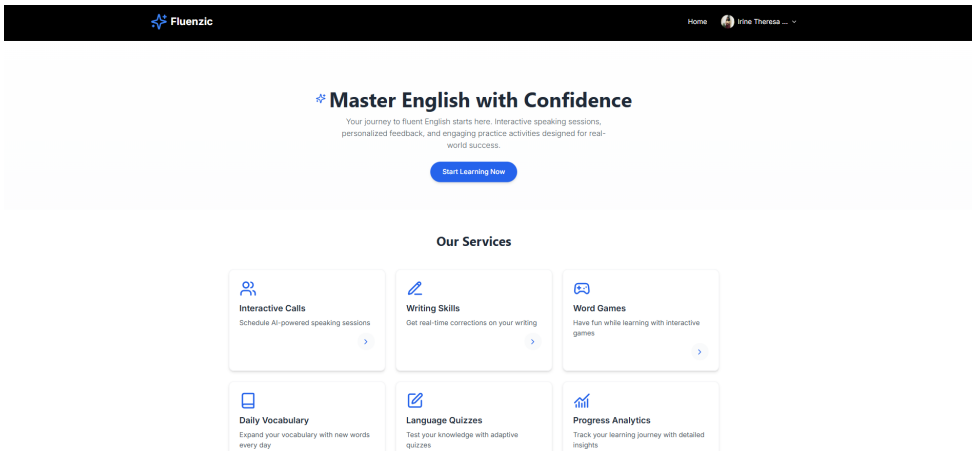


Figure B.3: User Home-Screen

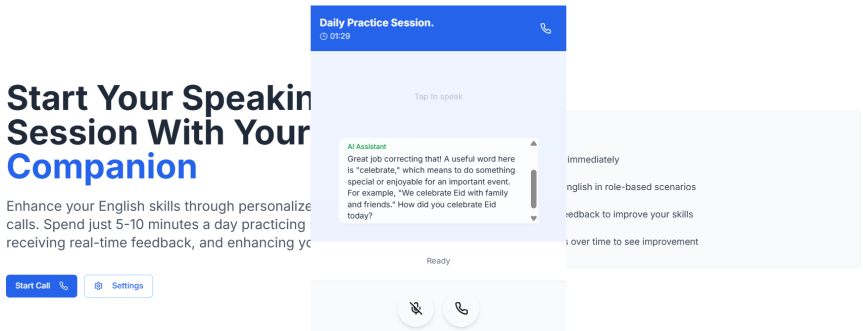


Figure B.4: Real-time Speech Recognition

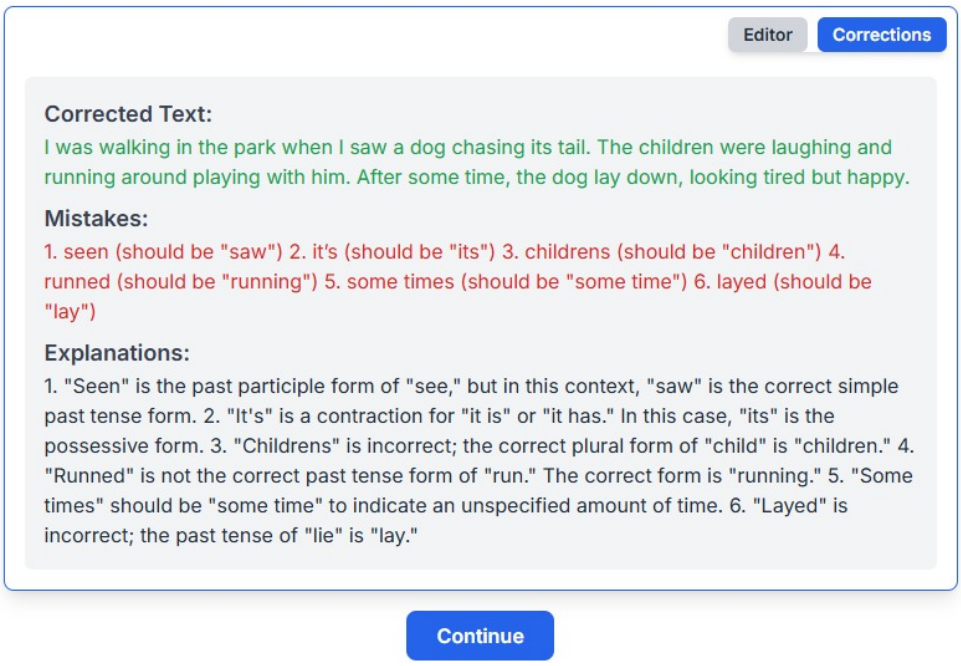


Figure B.5: Suggesitons Generated for Entered Sentence

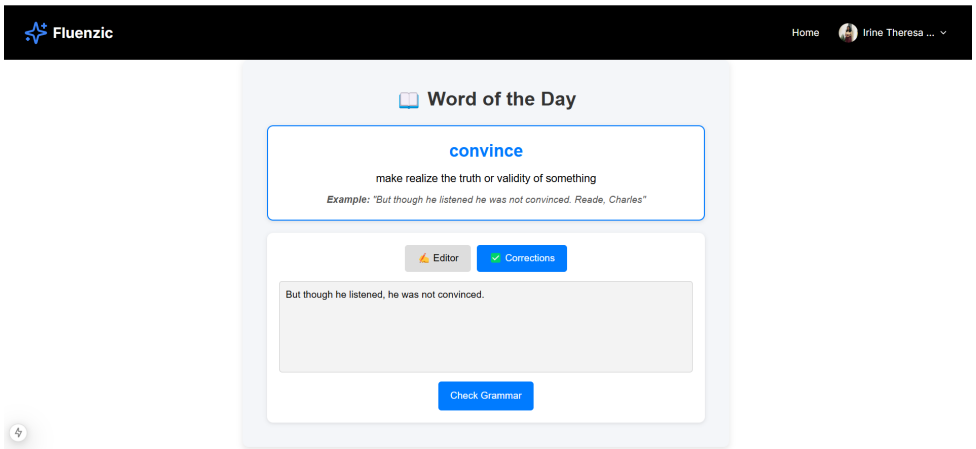


Figure B.6: Daily Vocabulary

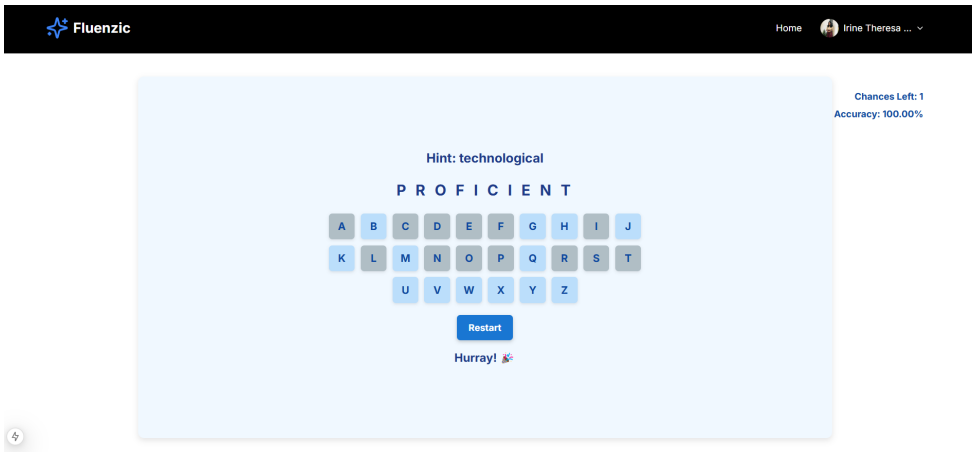


Figure B.7: Word Game - Synonym Generation by the Word2Vec Model

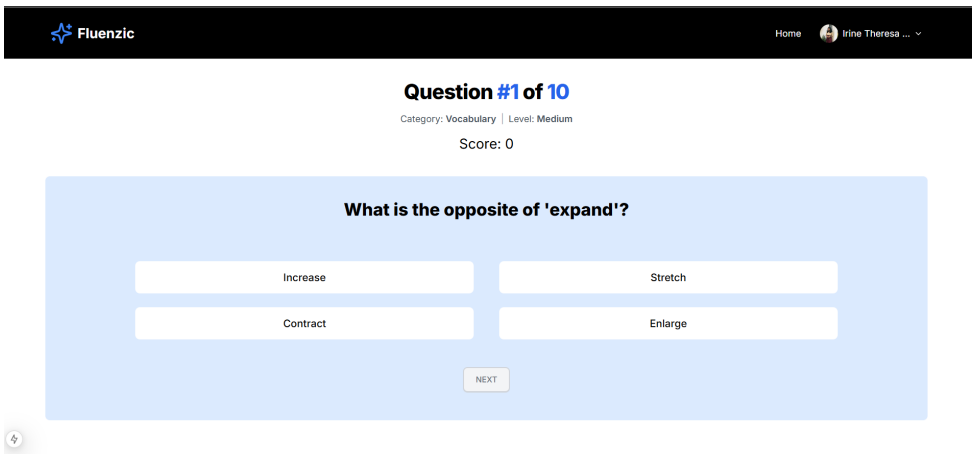


Figure B.8: Quiz Page

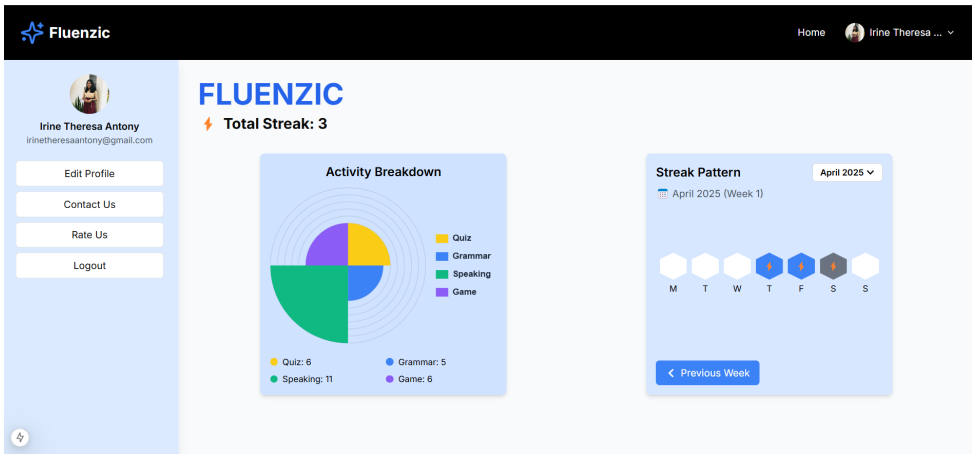


Figure B.9: Analytics Page

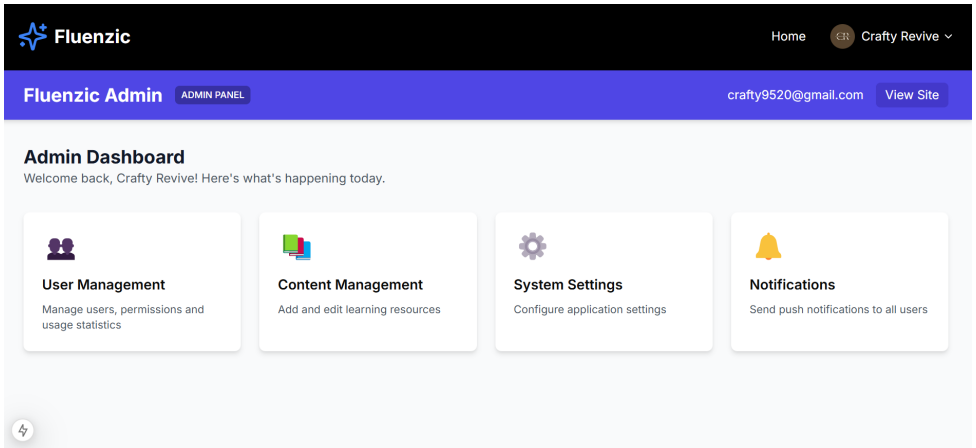


Figure B.10: Admin Dashboard

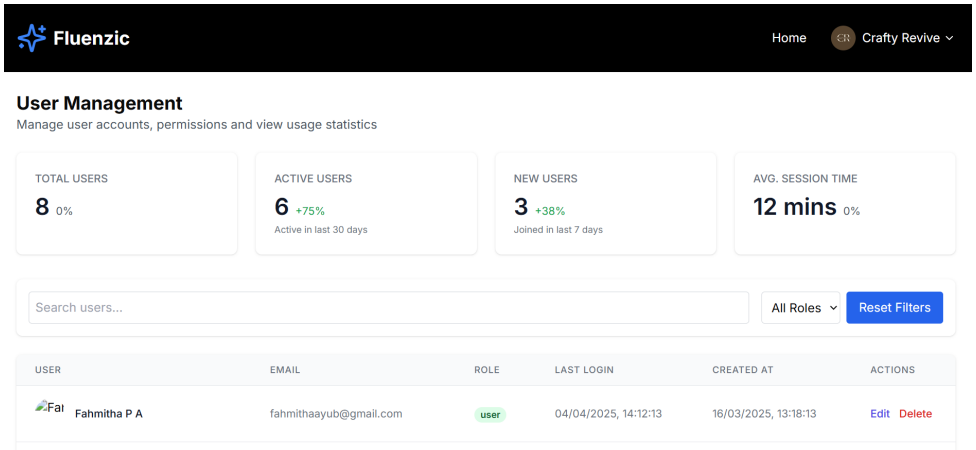


Figure B.11: User Management

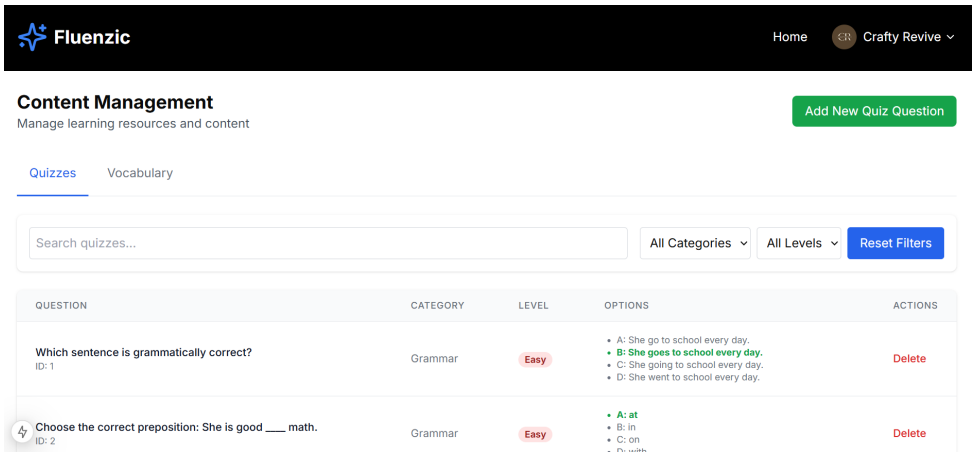


Figure B.12: Content Management

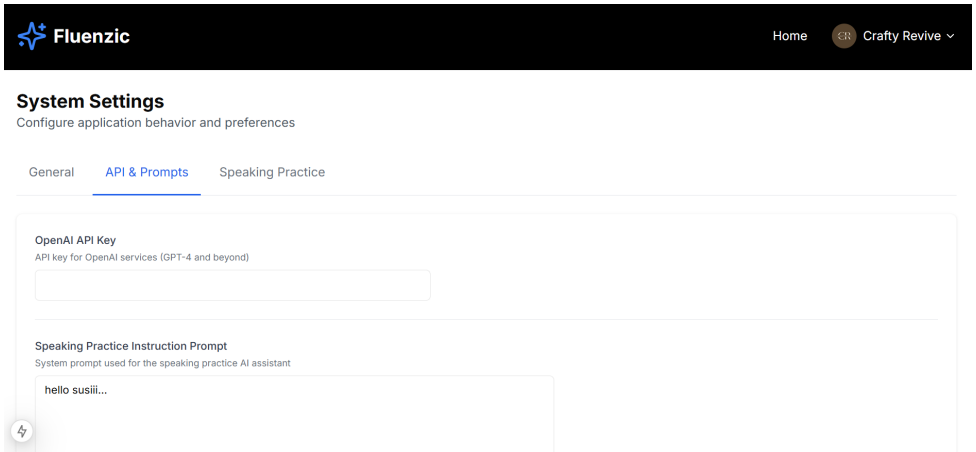


Figure B.13: System Settings

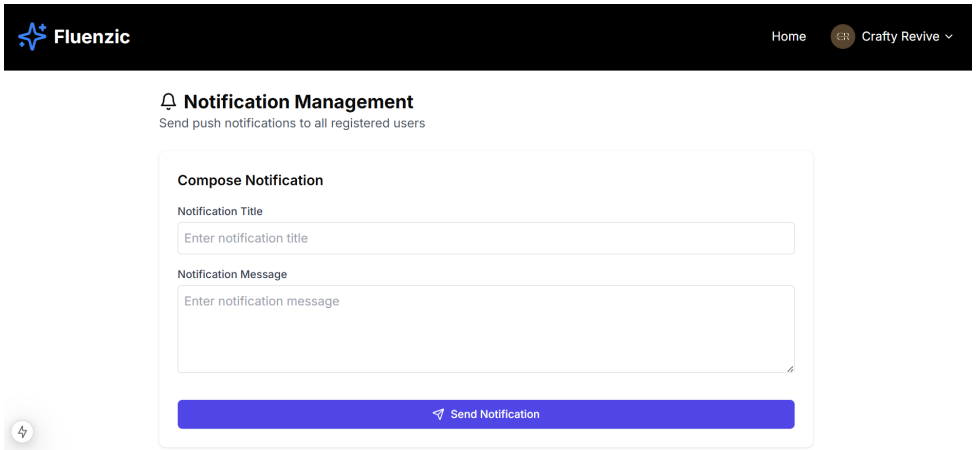


Figure B.14: Notifications Management